Acknowledgment

We, the researchers, sincerely extend our heartfelt gratitude to everyone who played a role in the successful completion of this thesis project, “Point-of-Sale and Inventory System for Pamela Mabulay Footwear Retail Store.”

Above all, we are deeply thankful to Almighty God for providing us with the strength, wisdom, and perseverance needed to accomplish this endeavor.

Our deepest appreciation goes to our thesis adviser, Professor Rogelio T. Plaza Jr. for your unwavering guidance, support, and insightful feedback throughout this project. Your expertise and patience were invaluable in shaping this system into a practical and efficient solution.

We are also profoundly grateful to Ms. Racquel Mejica and Ms. Nerissa Larino for sharing your valuable time, insights, and business expertise with us. Your contributions were instrumental in customizing our system to address real-world challenges effectively.

To our families and friends, we owe you a debt of gratitude for your constant encouragement, understanding, and support. Your belief in our abilities gave us the motivation to overcome the challenges we faced during this journey.

Finally, we extend our heartfelt thanks to every member of the team for your dedication, hard work, and teamwork in bringing this project to life.

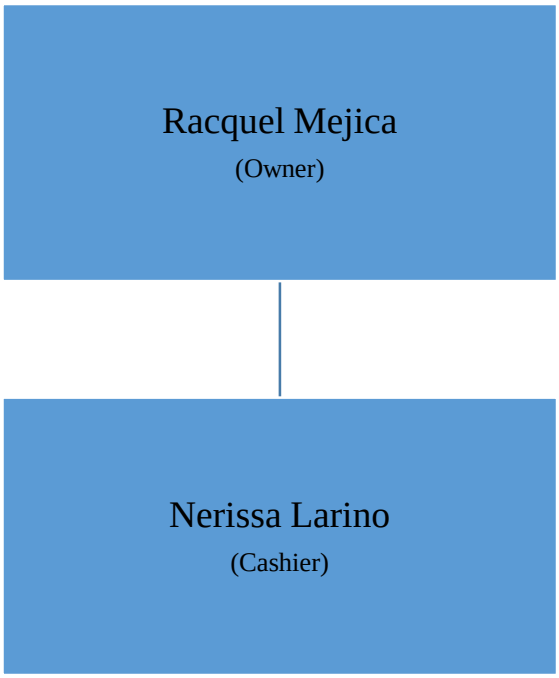
This accomplishment would not have been possible without the collective efforts, guidance, and support of everyone involved. Thank you for making this milestone in our academic journey possible.

Introduction

Pamela Mabulay Footwear Retail Store is well-known for providing a wide range of top-notch shoes. Located in between of Coromina Street and Severino Street in Recto Avenue, Quiapo, Manila, inside the bustling Cartimar Shopping Center. Throughout the years, the store has built a devoted clientele through their dedication to providing exceptional customer service and top-notch products. The store is facing major hurdles in its operating processes, despite achieving success. Currently, sales, inventory, and customer interactions are recorded manually in paper record books, a process that is becoming increasingly difficult, error-prone, and inefficient. This outdated technique doesn't just add complexity to the retailer's record-keeping, but also reduces the quality of the shopping experience.

To address these difficulties, the store has chosen to embrace a contemporary method by creating an offline Point of Sale (POS) system. This setup includes two primary elements: an administrative interface and a user interface, both intended for offline functionality. The administrative panel will improve operations by helping with effective inventory control, automated sales tracking, and detailed reporting. The User panel will improve the customer experience by enhancing customer service, expediting sales transactions, and simplifying product searches.

Organizational Chart



Background of the Study

Located in the Cartimar Shopping Center in Quiapo, Manila, Pamela Mabulay Footwear Retail Store is a key establishment in the community, offering a wide variety of quality shoes. Over the years, the store has cultivated a loyal customer base by consistently meeting their footwear needs. However, as with many small to medium-sized businesses, the store has begun to face operational challenges that are now impacting its overall productivity and service quality.

One significant issue is the store's reliance on manual processes to manage sales, inventory, and customer service records. Employees currently maintain physical log books to record transactions, inventory updates, and customer interactions. While this system has been effective in the past, it is increasingly inefficient and prone to errors, leading to delayed sales processing, inaccurate stock levels, and limited access to real-time data for decision-making.

As competition grows and customer demands evolve, the limitations of the current system are becoming more evident. To overcome these challenges, the development of a customized offline Point of Sale (POS) system is proposed. This new system will include an Admin panel for managing inventory, tracking sales, and generating reports, as well as a User panel for streamlining sales transactions and improving customer service efficiency.

History

Carlos, Timoteo, and Marina Basa founded Cartimar Shopping Center in the early 1970s, and it soon gained popularity as a place to buy specialty and reasonably priced foreign goods in Manila. It was wellknown for its pet stores, distinctive goods, and advantageous position in Recto, drawing in tourists, families, and students. Carimar is still a well-liked landmark today, praised for its affordable prices and diverse selection.

In 2021, Racquel Mejica opened the Pamela Mabulay Footwear Retail Store in Quiapo, Manila's busy Cartimar marketplace, which is situated along Recto Avenue. Serving sports fans and sneaker heads, the store specializes in reselling basketballs and rubber shoes. The business has established itself as a reliable destination for customers seeking fashionable and genuine shoes thanks to its carefully chosen assortment of premium and in-demand footwear. A major factor in drawing a consistent stream of clients and fostering the store's increasing success in the cutthroat footwear reselling industry has been its prime position in one of Manila's busiest business districts.

Mission

To provide high-quality, authentic basketball and rubber shoes to our customers by offering a curated selection of sought-after footwear at competitive prices, ensuring exceptional customer satisfaction. We strive to create a trusted shopping experience that caters to the needs of athletes, sneaker enthusiasts, and everyday customers, while upholding integrity and passion for the footwear industry.

Vision

To become the leading footwear reseller in Manila, renowned for our commitment to quality, authenticity, and customer satisfaction. We aim to expand our reach, inspire a community of loyal customers, and contribute to the vibrant sneaker culture by continuously delivering value and innovation in the footwear market.

Statement of the Problem

The researchers found out problems in the day-to-day manual process of Pamela Mabulay Footwear Retail Store, these are the problems they gathered after the interview:

- Labor Intensive: Recording sales, updating inventory, and generating reports manually is labor-intensive and time-consuming, leading to inefficiencies in daily operations

- Inventory Issue: Stock outs, overstocking, missed sales, and increased expenses are all consequences of poor inventory management, which can also result in imprecise stock levels, poor visibility, and delayed decision-making.

- Limited Reporting Capabilities: Manual systems typically lack advanced reporting features, making it difficult to generate detailed and timely reports on sales performance, inventory levels, and financial metrics.

- Receipt Issuance: Writing receipts by hand can be ineffective and exhausting.

- Slow checkout process: The absence of an automated point-of-sale system may result in problems and delays while processing customer payments

Objectives of the Study

These objectives outline the key goals of the proposed system, focusing on addressing common challenges in inventory management, such as inaccuracies, limited access to real-time data, reporting limitations, poor stock visibility, and the potential for human error

- To propose a system that automates inventory management, sales recording, and report generation in order to increase operational effectiveness, minimize human error, and cut down on the time and labor needed for everyday tasks.

- To propose a system with built-in inventory management system, that aims to lower stock outs, overstocking, missed sales, and costs by monitoring stock levels, automating alerts and granting real-time access.

- To propose a system with advanced reporting features, offering customizable and detailed reports on sales performance, inventory levels, and financial metrics.

- To propose an automated receipt issuance system that streamlines the checkout procedure and boosts overall operational efficiency by doing away with the inefficiencies and weariness related to writing receipts by hand.

- To propose an automated point-of-sale system that speeds up the checkout process, reduces delays, and enhances the efficiency of customer transactions.

Review of Related Literature

Local Literature

According to Atilano-Tang, Damsani & Ministry of Public Works-Basilan District Engineering Office (2023), effective inventory management is critical to the success of public organizations, as it directly impacts service delivery and operational efficiency. Numerous studies have examined inventory management practices in the public sector, highlighting the urgent need for improved systems and processes to manage inventory more effectively.

According to Mendoza, Santos, Balbuena, Cabral, & Agustin (2019), point of sale (POS) systems are enterprise solutions designed to manage sales activities and inventory simultaneously. These systems provide a comprehensive approach by recording transaction details, including client information, purchased items, prices, and dates, all in one place. POS systems are particularly valuable for manufacturers and retailers as they facilitate fast, efficient service, allowing businesses to handle high volumes of transactions with ease. The streamlined data management offered by POS systems simplifies monitoring and enhances operational efficiency.

According to Mendoza, Santos et al (2019), combining an Inventory System with a POS System offers a powerful solution for managing and reviewing business transactions. This integrated approach not only speeds up the process but also increases reliability compared to manual methods. An integrated system reduces the likelihood of updating errors and provides easy access to accurate data. With real-time updates and comprehensive information, management can make informed decisions quickly and consistently, thereby improving overall business operations.

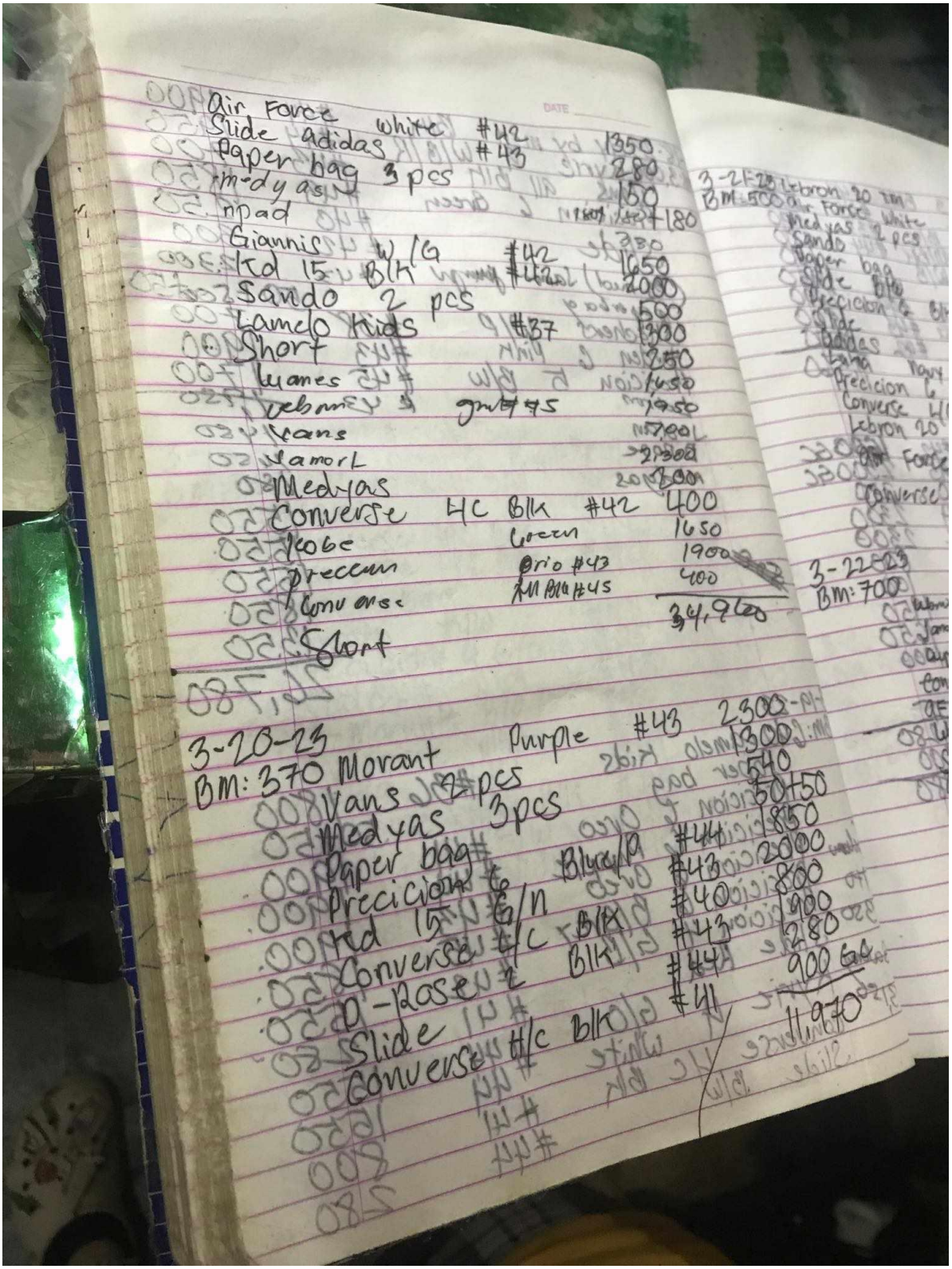
Foreign Literature

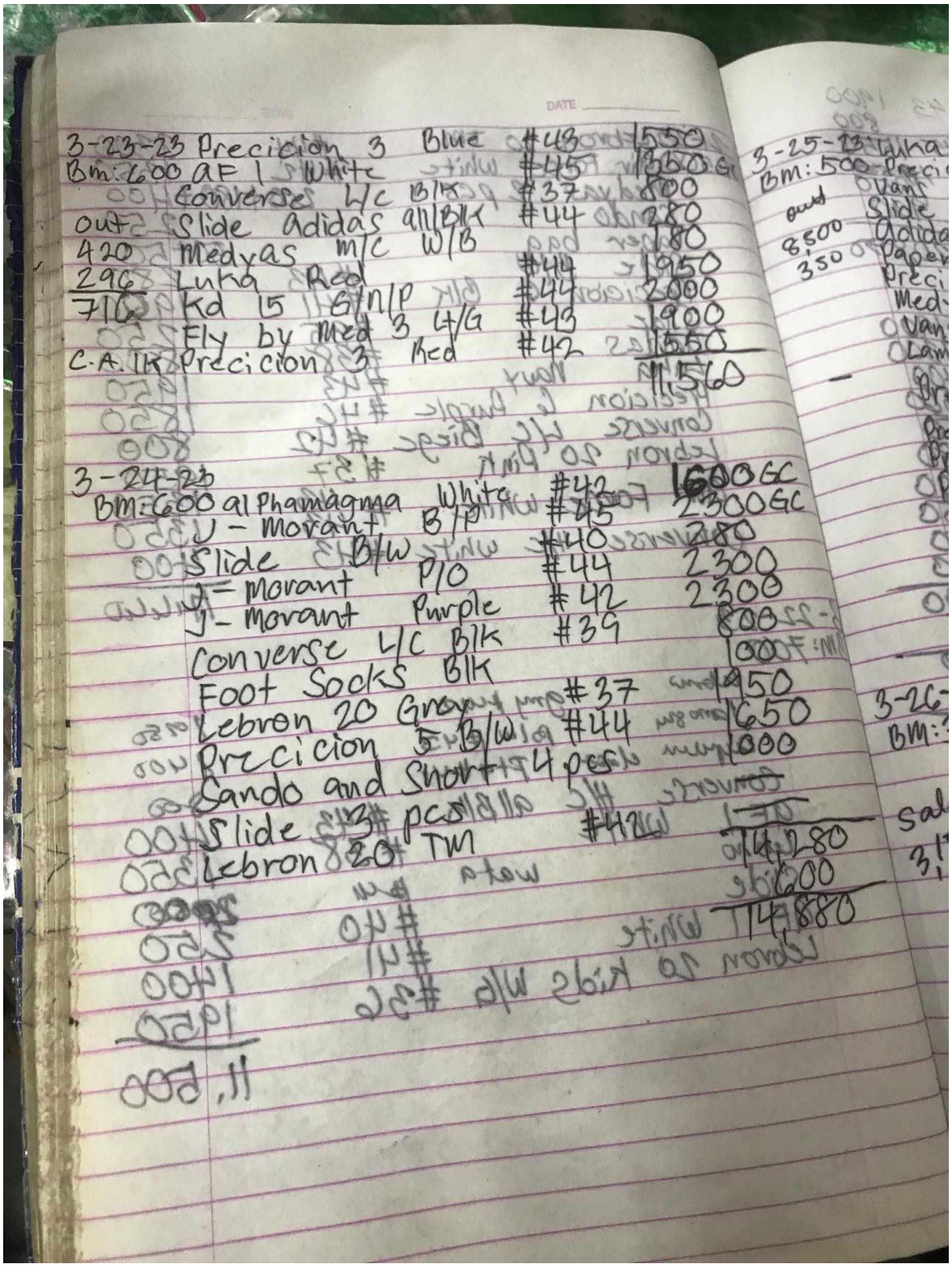
According to Deshmukh & Tak (2022), the integration of Inventory Management Systems (IMS) is crucial for the success of retail businesses, particularly in the footwear industry. These systems are essential tools utilized by a diverse range of companies, organizations, and institutions to manage their stock effectively. An IMS tracks purchase orders and maintains inventory levels, helping businesses keep up with the high volumes of inventory that flow in and out daily. As access to resources expands and the demand for products increases exponentially, manual inventory management becomes increasingly impractical.

According to Nilesh Waghmare, Sachin Chavan (2022), point of sale software gives business owners a convenient way of finding out customers and of recording sales. It can keep a record of the shop inventory, updating it when an order is processed. It can even print out receipts; perform credit card processing, track customers, etc. Point of sale software eases the flow at checkout terminals while recording all the data which will assist you to make better business decisions. Point of sale software allows users to input via keyboard or mouse, and a few even have a touch screen interface. Point of sale software can even help with credit card processing.

According to Edward B. Panganiban, Jenefer P. Bermusa (2020), computerized inventory system starts with Point-of-Sale (POS) system. POS system serves as an aid for the regular monitoring of the products, the status of the orders, the percentage of stocks available, and warehouse inventory. More research shows that a POS device streamlines the process of loading inventory into a database after transactions have been completed, allowing companies to manually handle inventories. POS information can also be used to produce projected sales trends based on prior request. This could affect work orders which "should then be measured as to how much customers are likely to expect, so POS results can be used to anticipate what consumers will purchase.

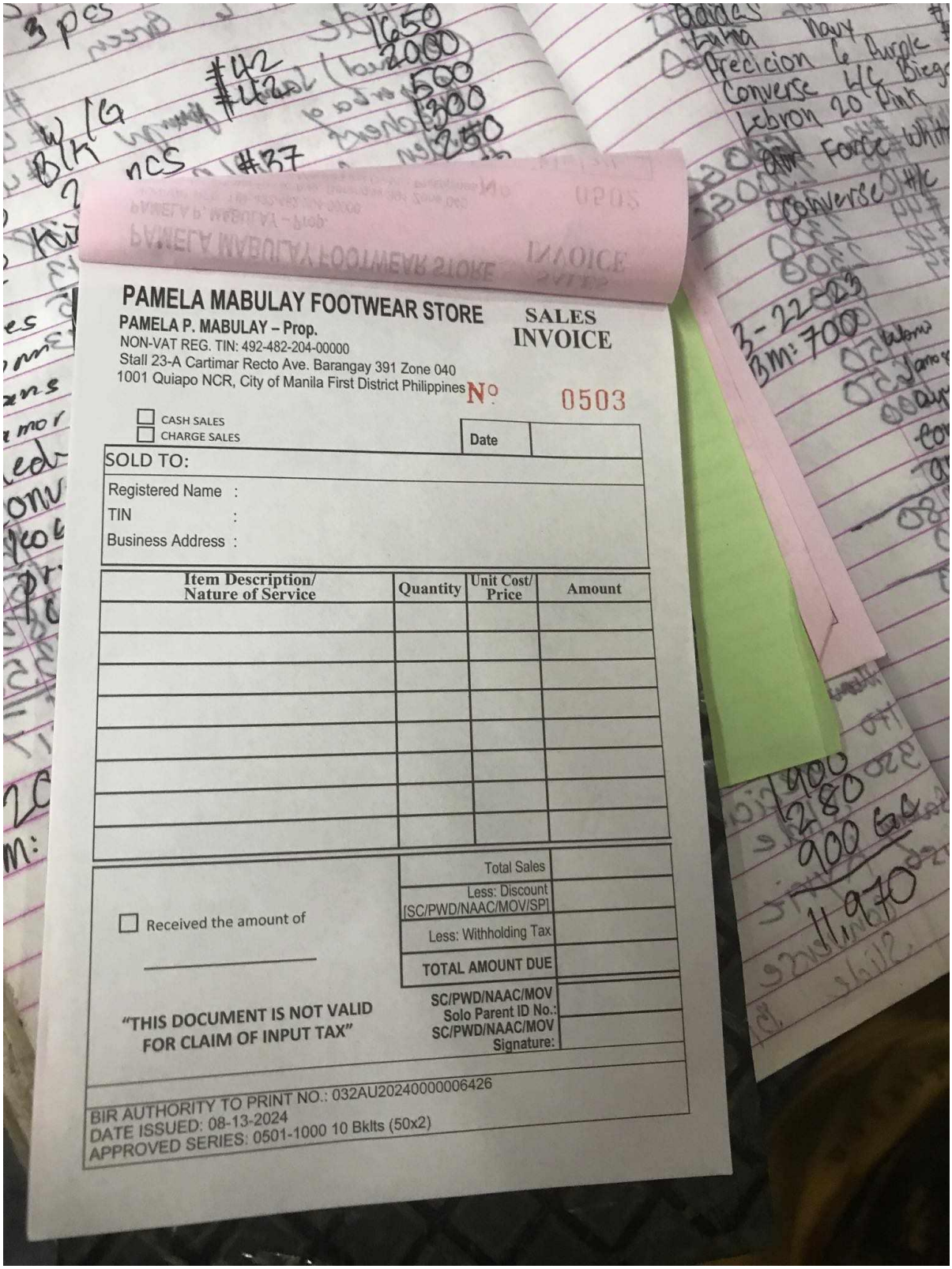
Old System Screenshot





3-23-23 Precicion 3 Blue #43 1550
Bm: 600 AF 1 White #45 1350 GC
Converse L/C B/K #37 800
out Slide Adidas all B/K #44 280
420 Medyas m/c W/B 180
296 Luka Red #44 1950
716 Kd 15 G/n/P #44 2000
Fly by Med 3 L/G #43 1900
C.A.K Precicion 3 Red #42 1550
11,560

3-24-23
BM: 600 al Phamagma White #43 1600 GC
J - Morant B/P #45 2300 GC
Slide B/W #40 280
J - Morant P/O #44 2300
J - Morant Purple #42 2300
Converse L/C Blk #39 800
Foot Socks Blk 100
Lebron 20 Gray #37 1950
Precicion 5 B/W #44 1650
Sando and Short 4 pcs 1000
Slide 3 pcs #42 1400
Lebron 20 TW 14,280
1600
14,880



PAMELA MABULAY FOOTWEAR STORE
PAMELA P. MABULAY – Prop.
NON-VAT REG. TIN: 492-482-204-00000
Stall 23-A Cartimar Recto Ave. Barangay 391 Zone 040
1001 Quiapo NCR, City of Manila First District Philippines

SALES INVOICE

No. 0503

☐ CASH SALES
☐ CHARGE SALES

Date

SOLD TO:

Registered Name :
TIN :
Business Address :

Item Description/ Nature of Service	Quantity	Unit Cost/ Price	Amount

☐ Received the amount of

_____

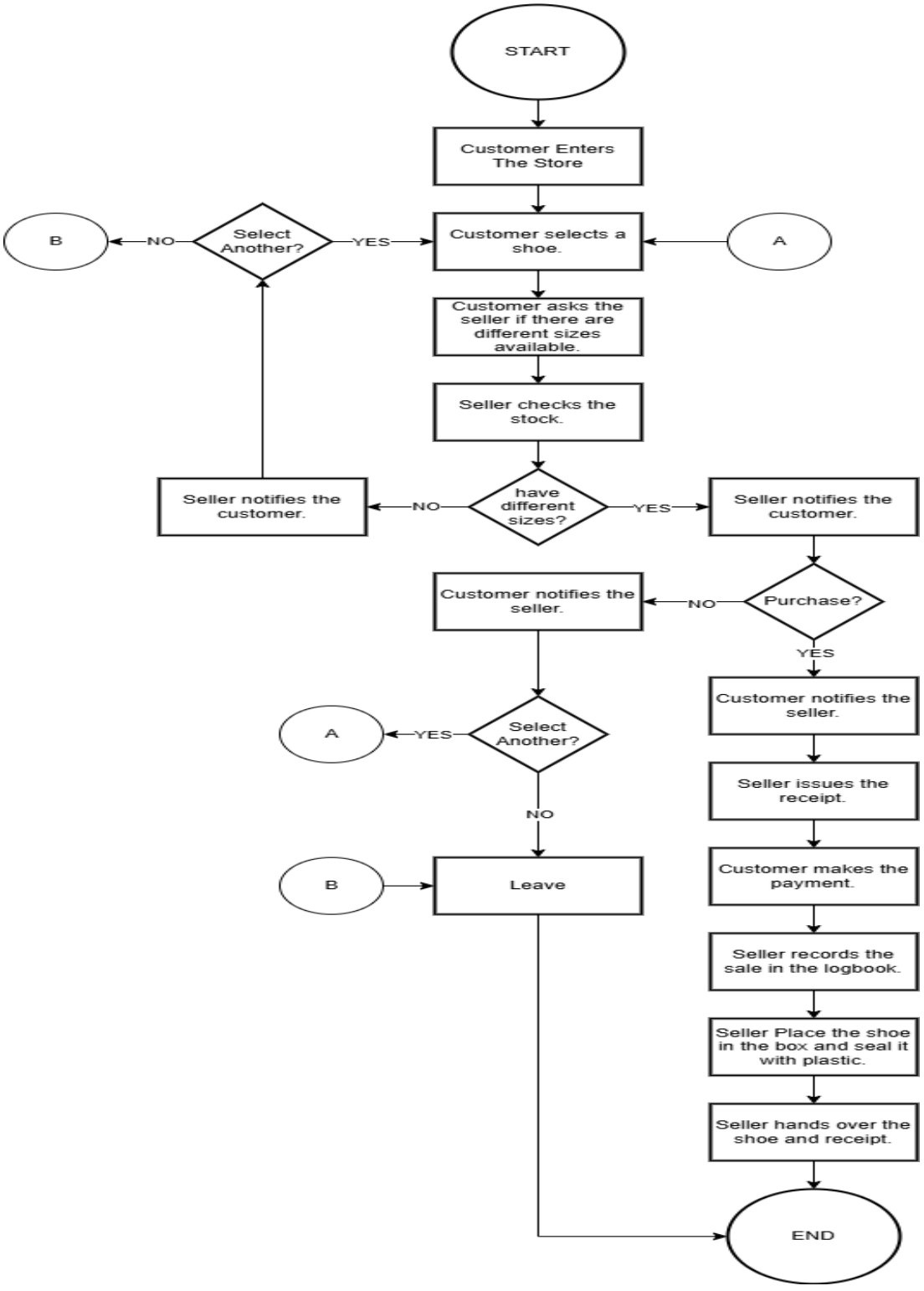
"THIS DOCUMENT IS NOT VALID FOR CLAIM OF INPUT TAX"

Total Sales	
Less: Discount [SC/PWD/NAAC/MOV/SP]	
Less: Withholding Tax	
TOTAL AMOUNT DUE	
SC/PWD/NAAC/MOV Solo Parent ID No.:	
SC/PWD/NAAC/MOV Signature:	

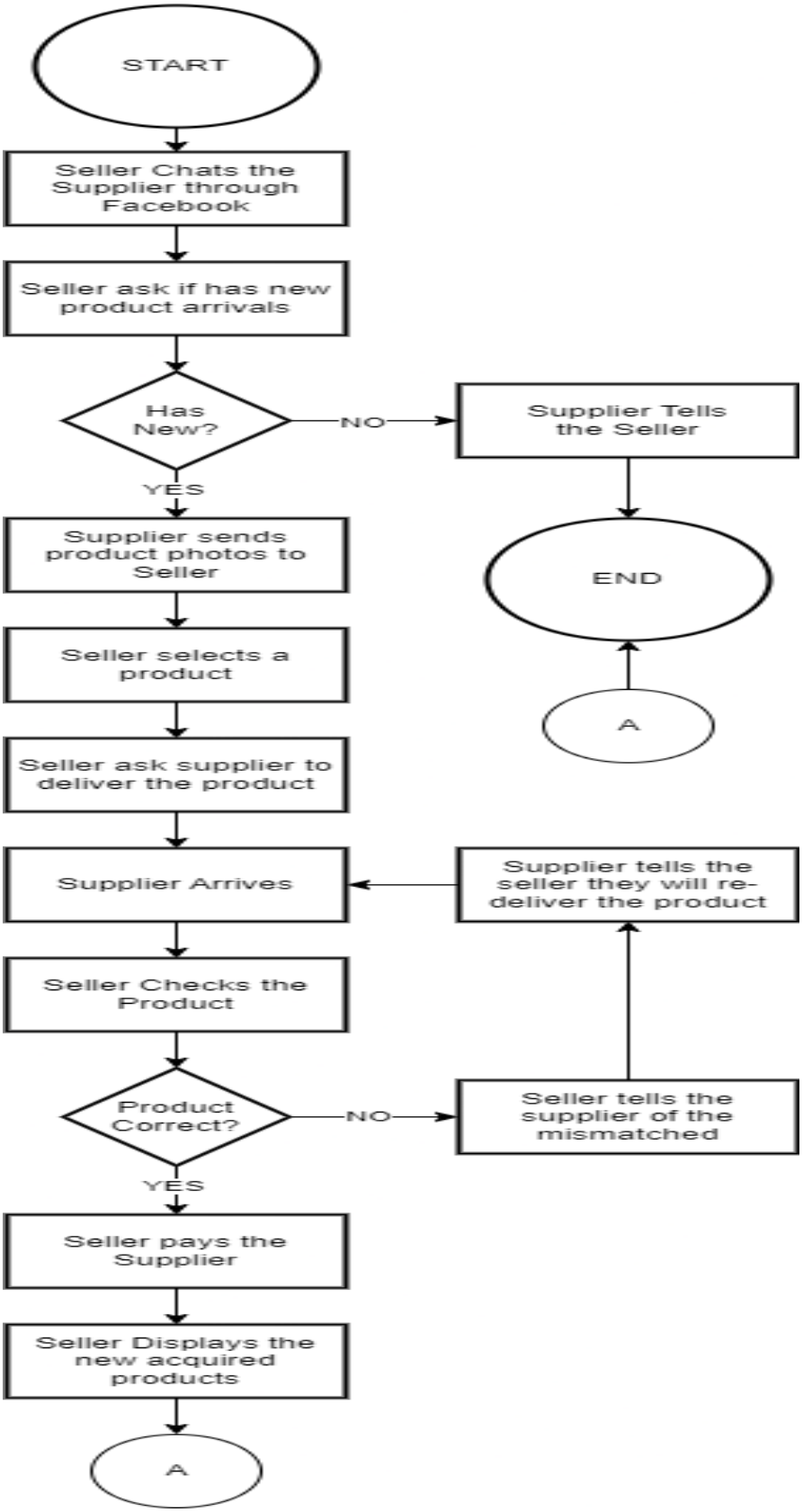
BIR AUTHORITY TO PRINT NO.: 032AU20240000006426
DATE ISSUED: 08-13-2024
APPROVED SERIES: 0501-1000 10 Bklts (50x2)

Old System Flowchart

Sale, Receipt and Sale Record

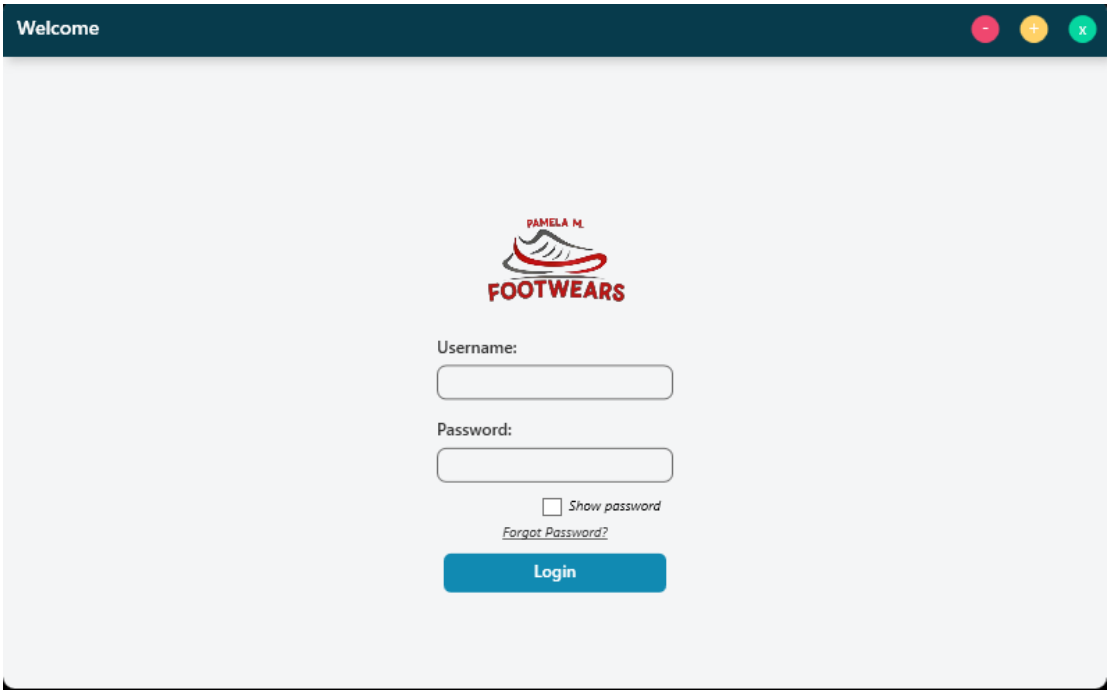


Supplier and Displaying Product

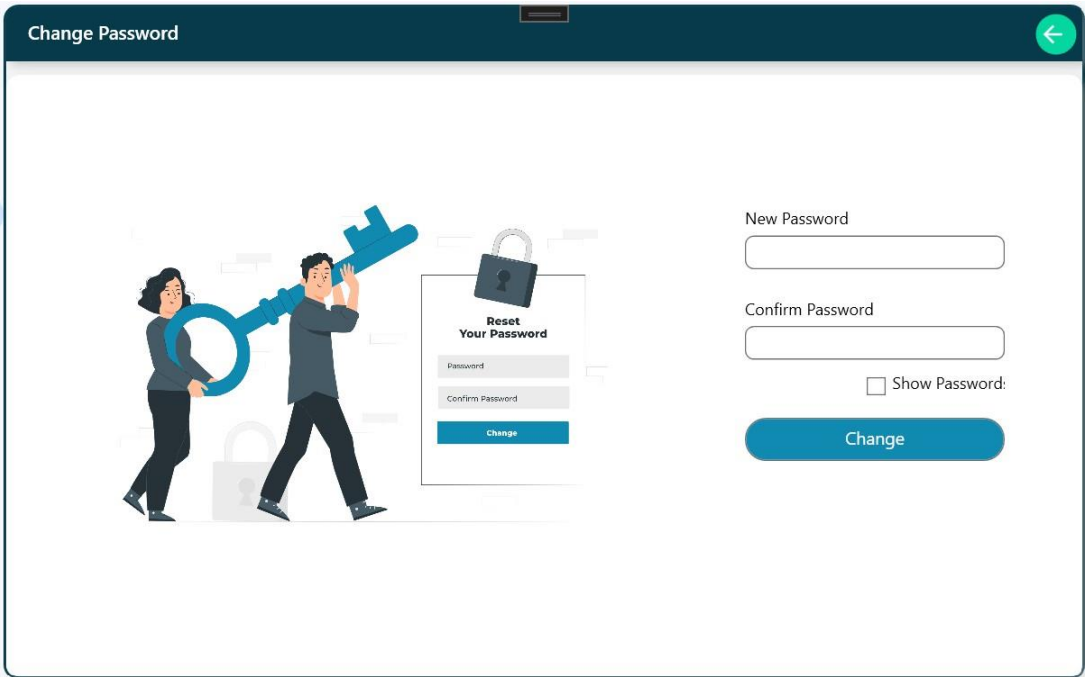


New System Screenshot

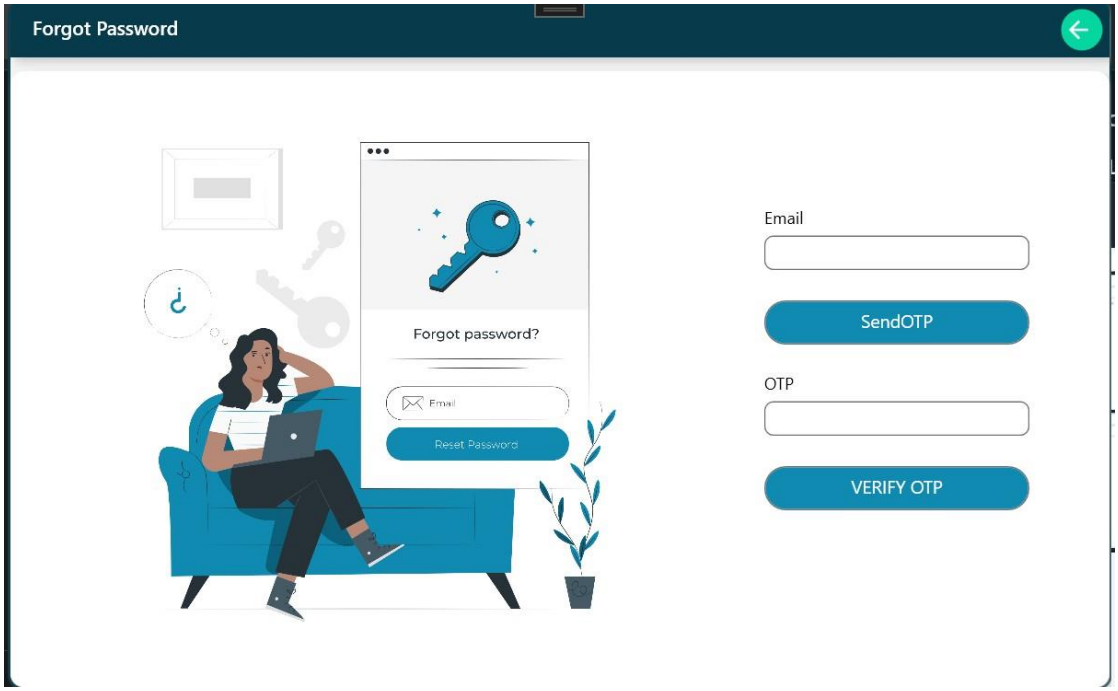
Login



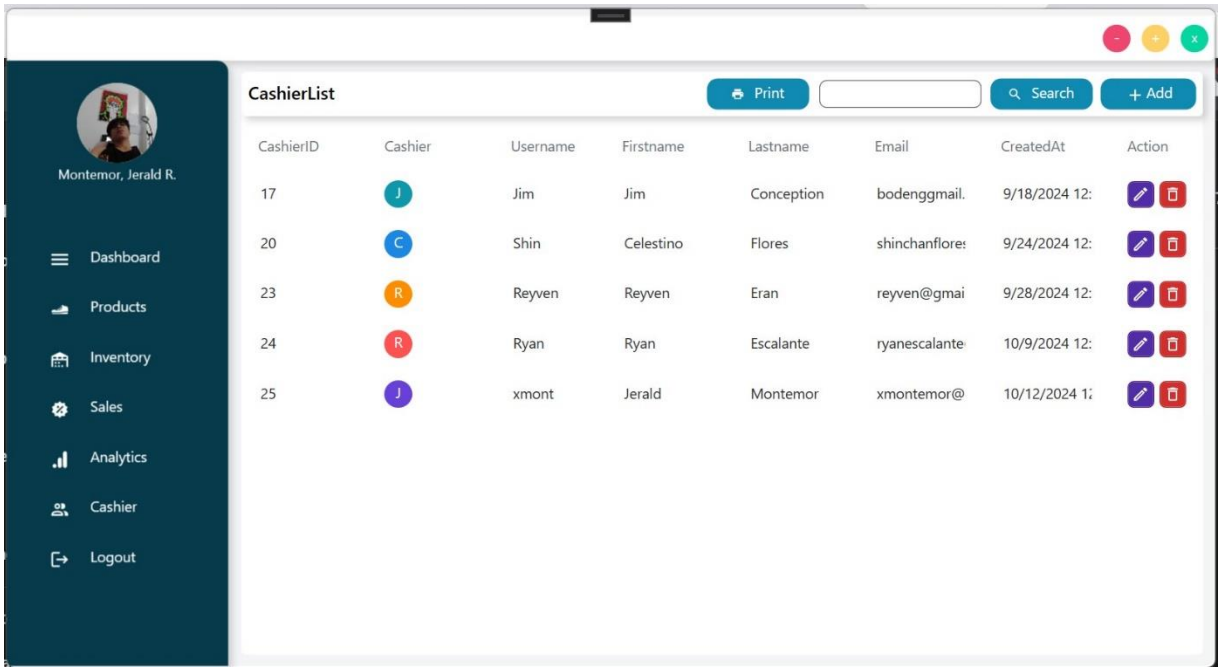
Change Password



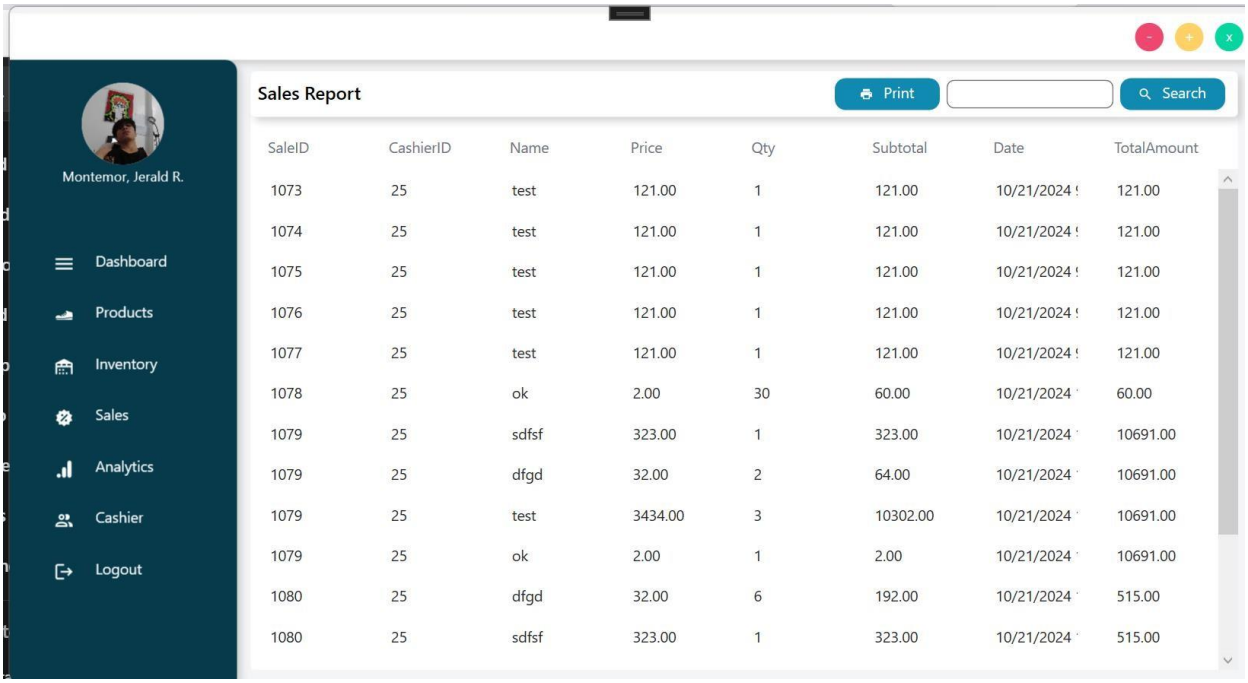
Forgot Password



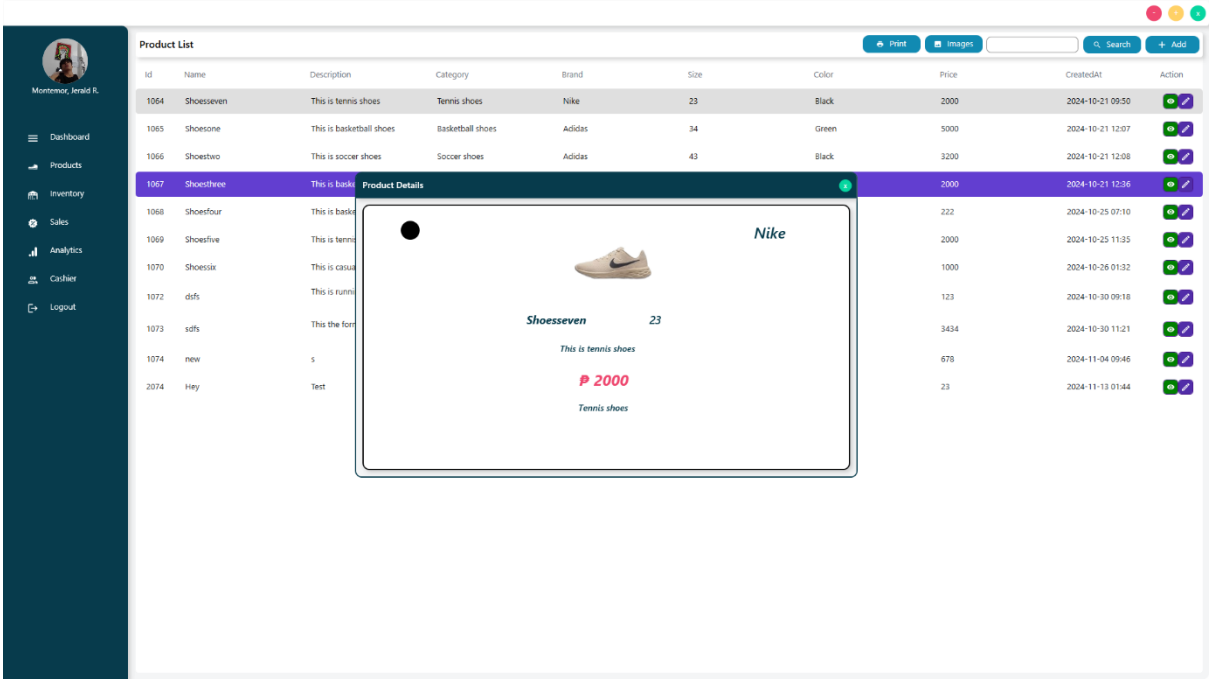
Manage Cashier



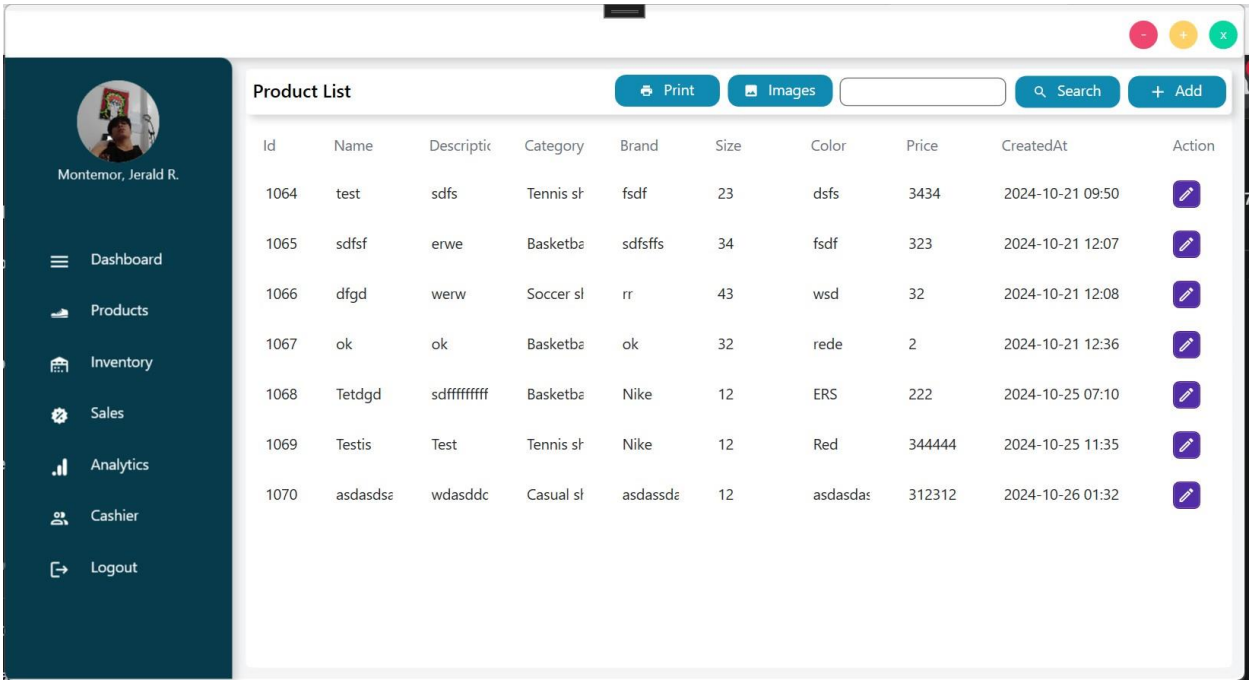
Sales Report



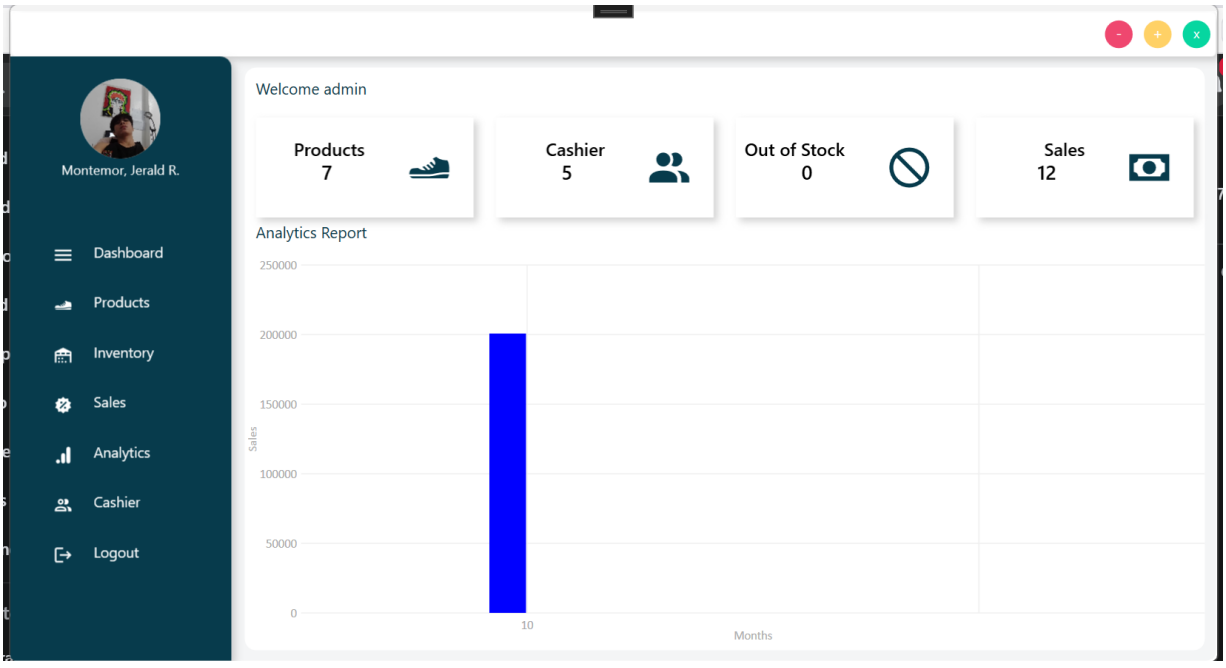
Product Inventory



Product Information

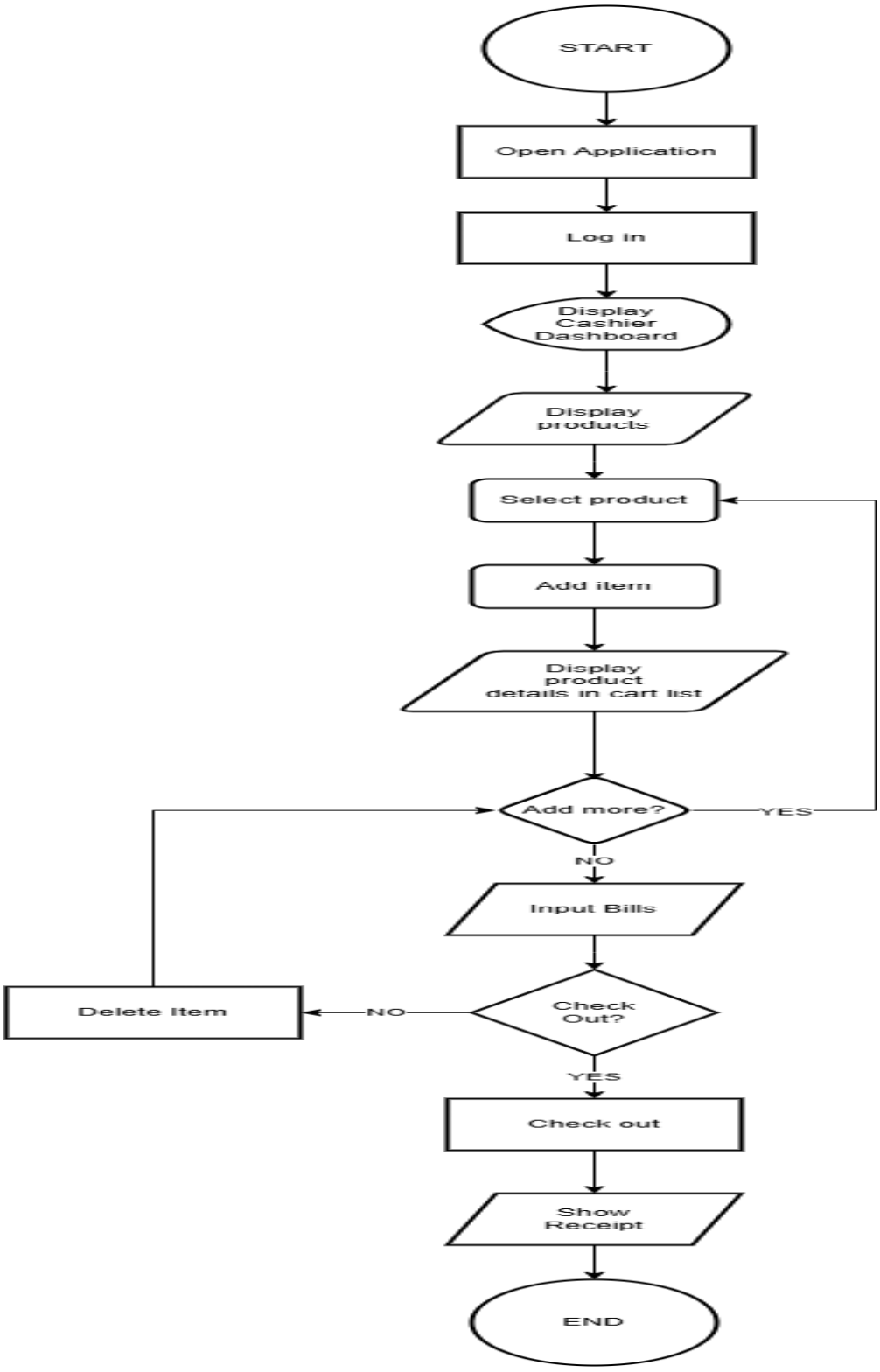


Dashboard

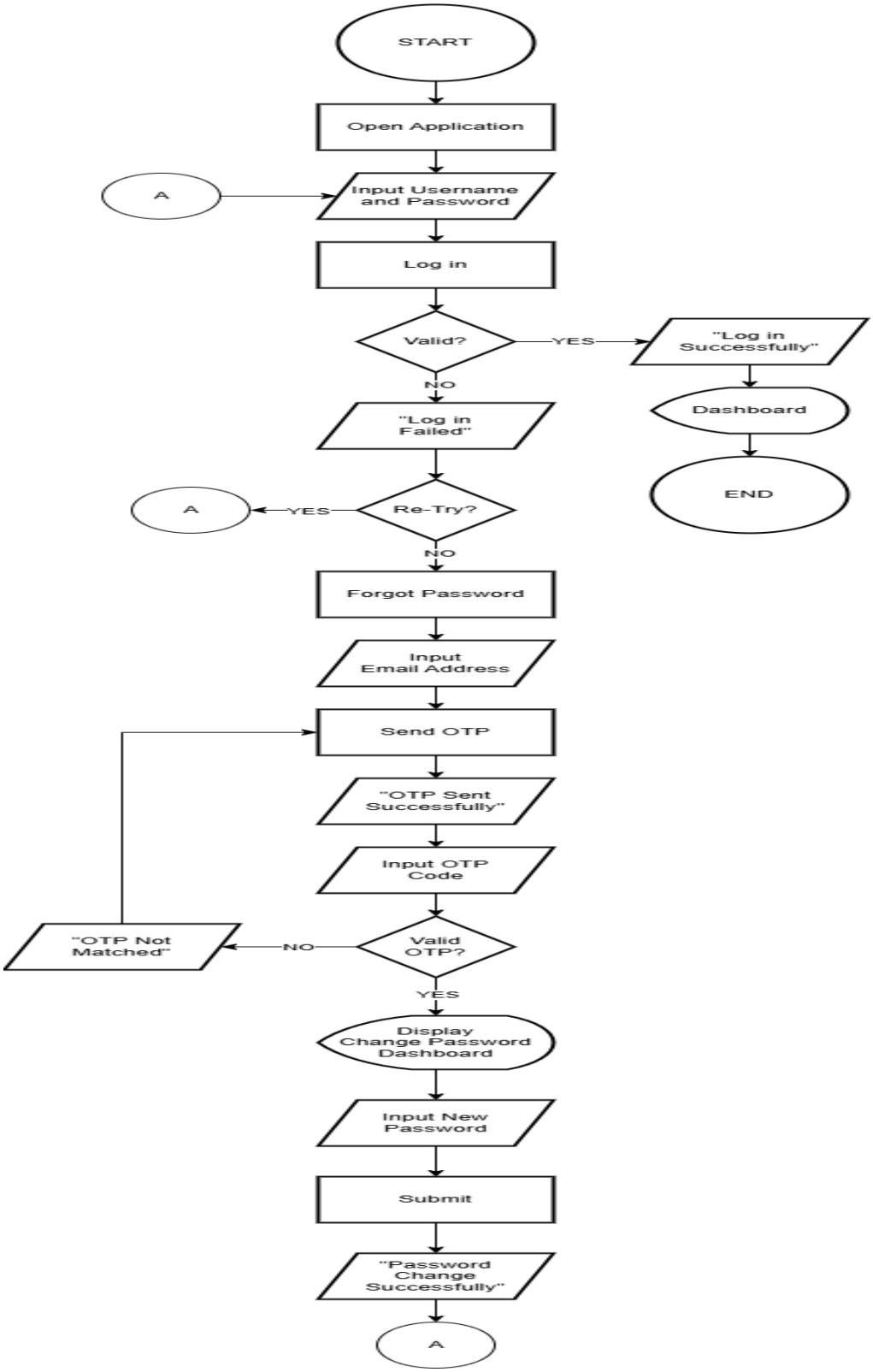


Proposed System Flowchart

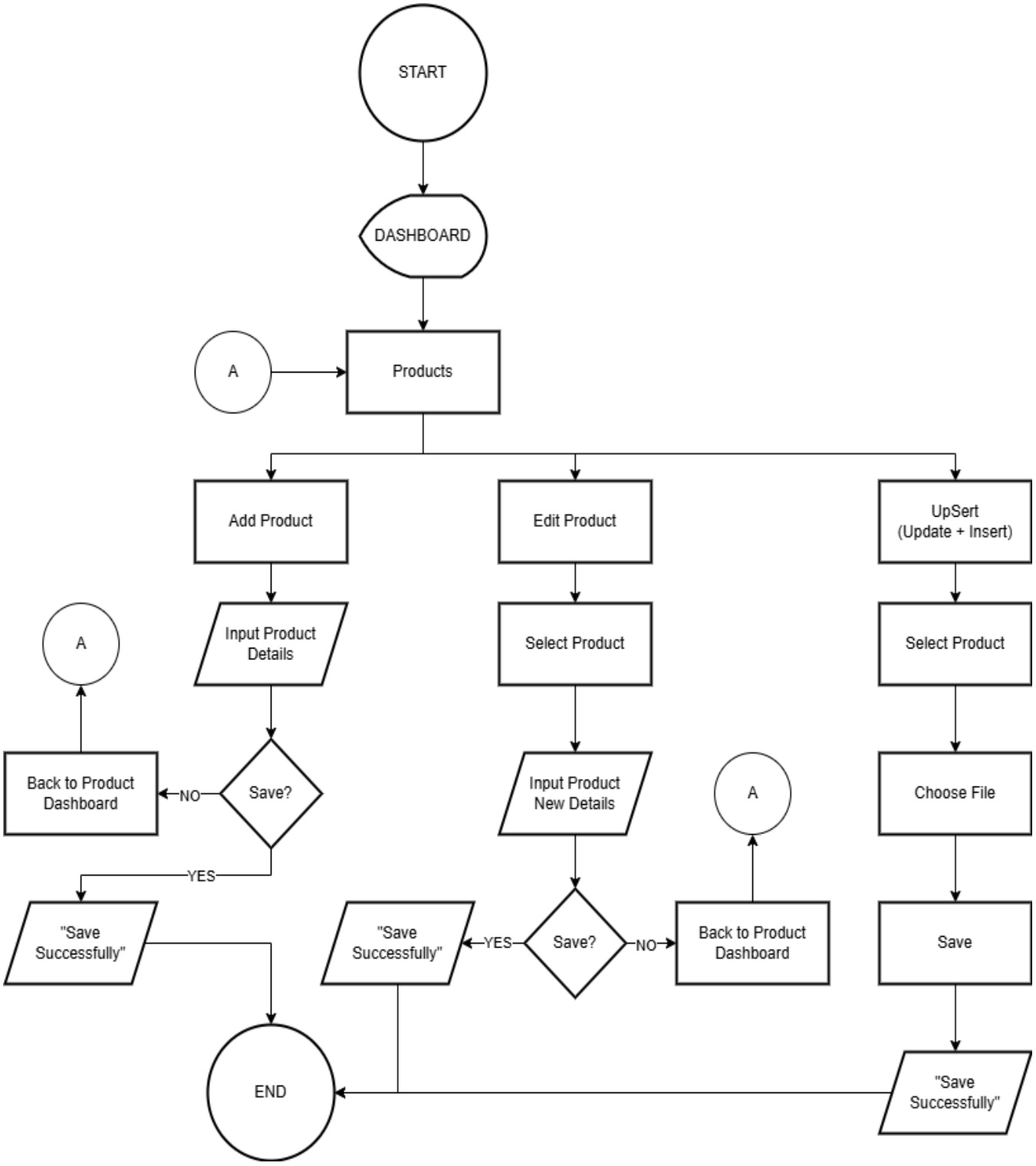
Cashier Dashboard Flowchart



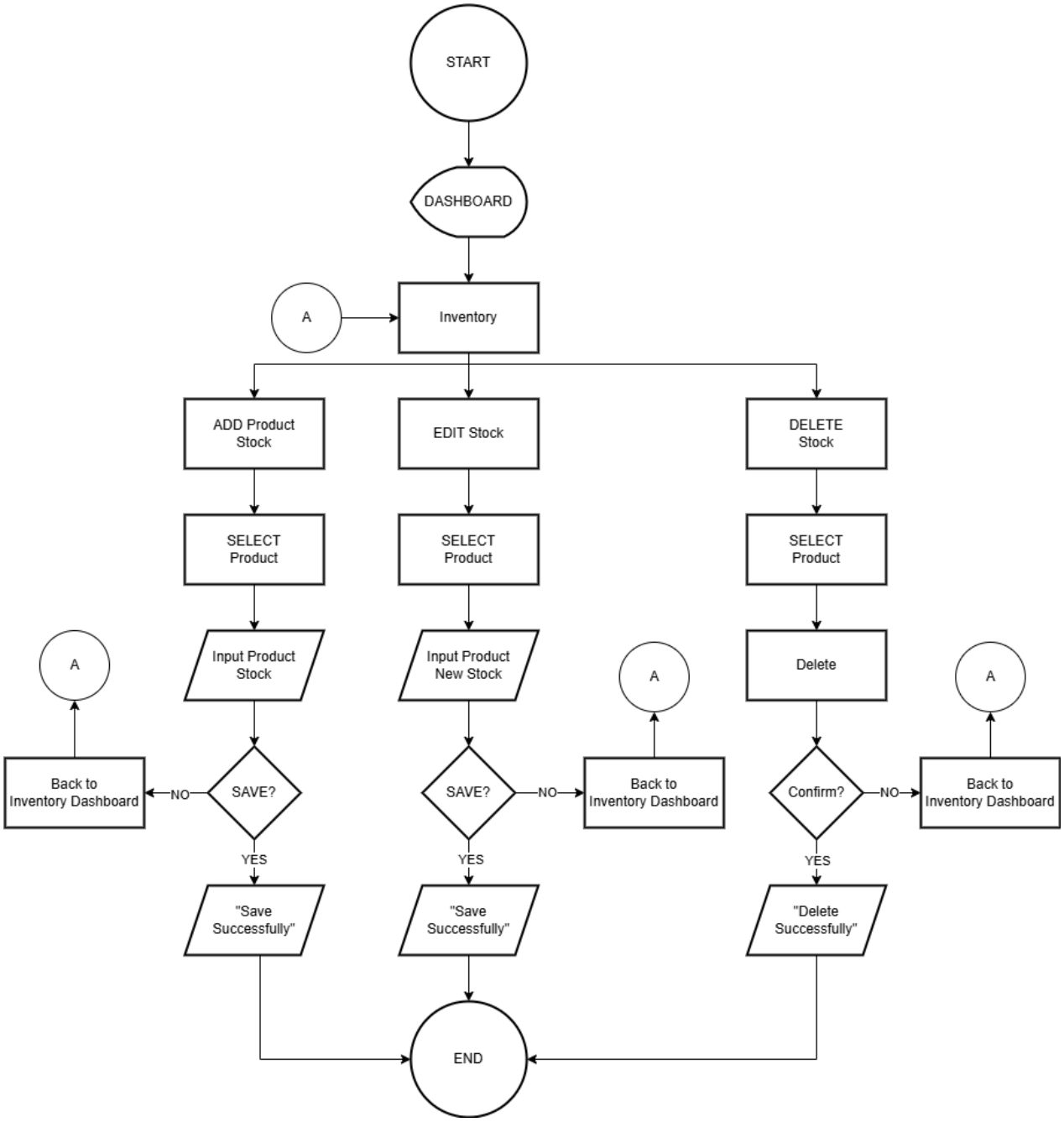
Login/Forgot Password Flowchart



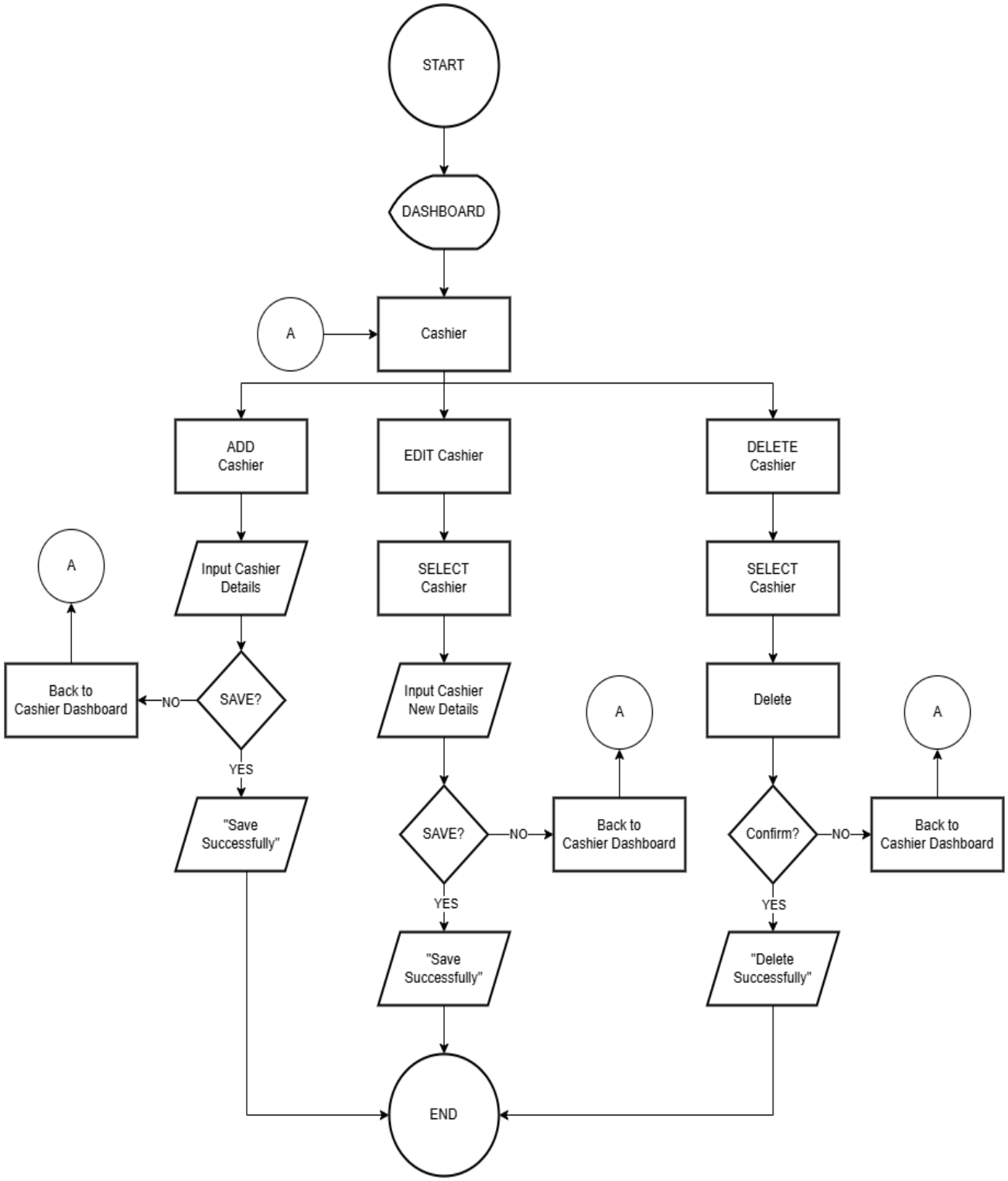
Product Flowchart



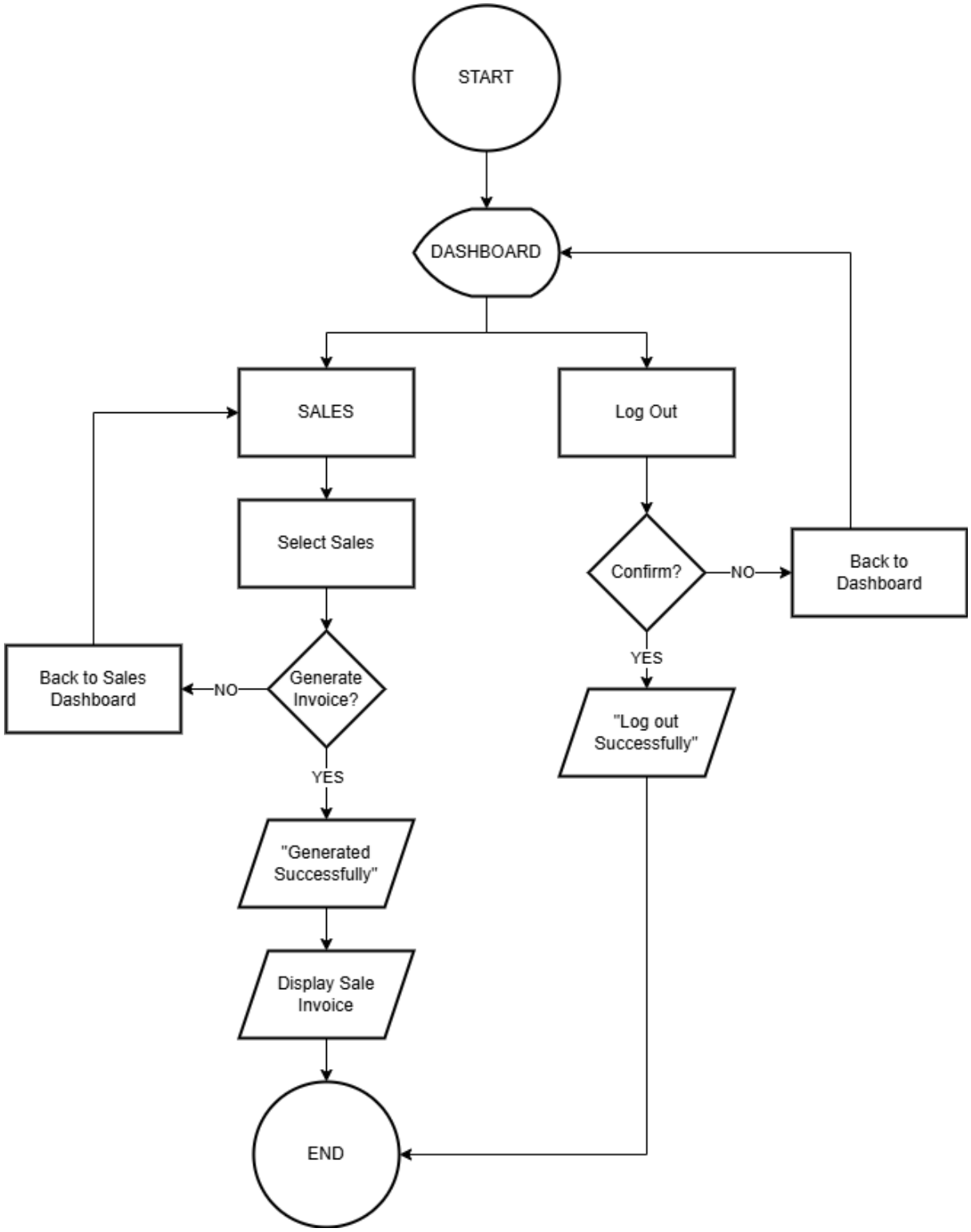
Inventory Flowchart



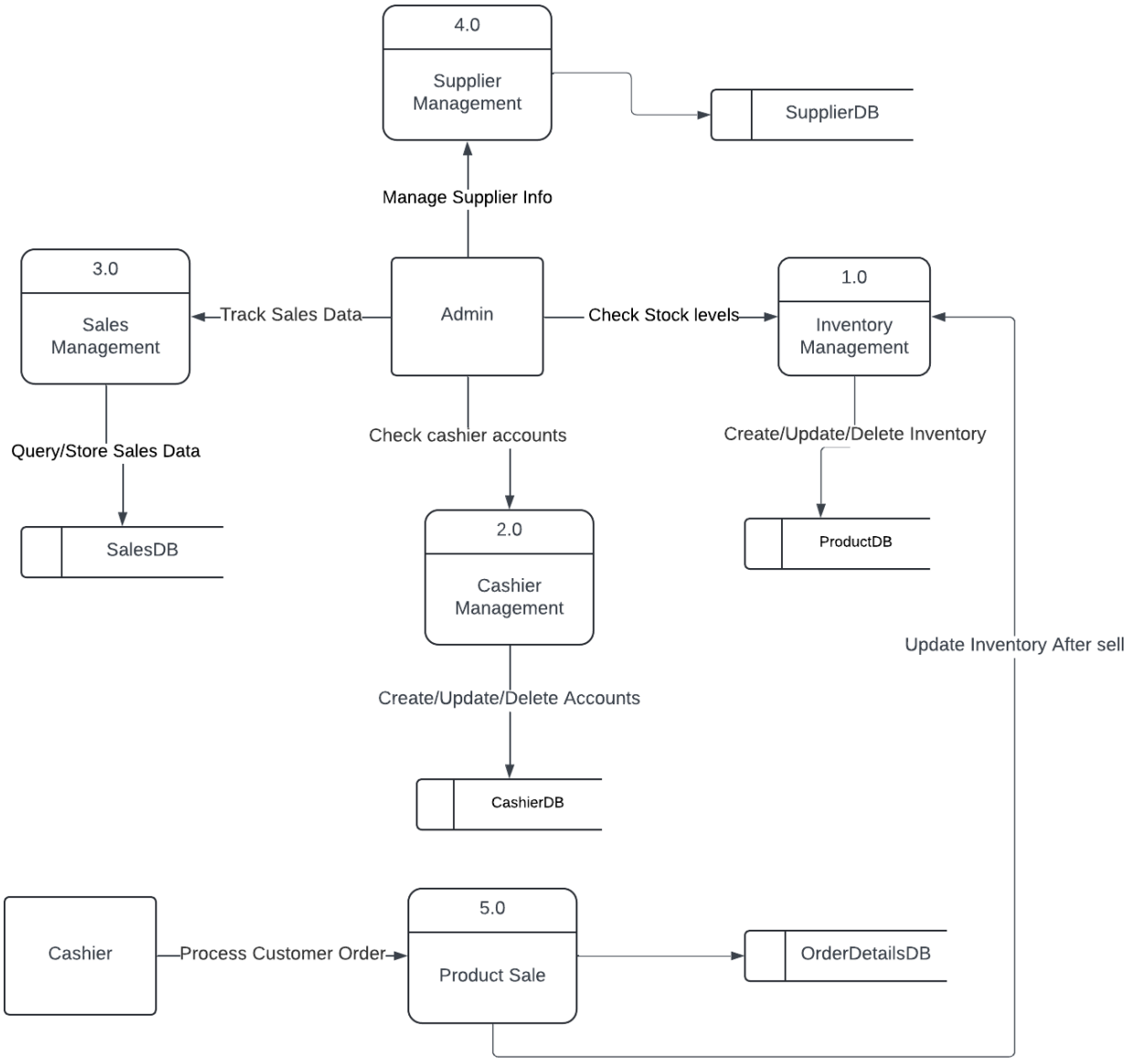
Manage Cashier Flowchart



Sales and Logout Flowchart



Data Flow Diagram



Definition of Terms

Clientele - It refers to the people who regularly buy from or use a business's services

Demands - The consumer's desire to purchase a particular good or service.

Evident – Easily seen or understood, Obvious.

Expediting - to make something more quickly.

Inventory - A complete list of items such as property, goods in stock, or the contents of a building.

Labor intensive - A process that requires a large amount of labor to produce its goods or services.

Overcome - Succeed in dealing with a difficulty.

Point of sale - The place where a customer and a business complete transaction.

Retailer - A person or business that you purchase goods from.

Top notch - A very high standard.

System Requirements

Hardware Requirements

Minimum:

- Memory: 4GB RAM
- Storage: 300MB Available Disk Space
- Processor: Intel Core I3 8th Gen, AMD Athlon (or Equivalent) 

Recommended:

- Memory: 8GB RAM
- Storage: 1 GB Available Disk Space
- Processor: Intel Core I5 13th Gen or Higher, AMD Ryzen 5 5600 or Higher

Software Requirements

- OS: Windows 10 (64-bit) or Higher 
- .Net Framework 4.8 or later 
- Microsoft SQL Server 2022 
- SQL Server Management Studio 

Software and Hardware Used

Hardware

- Device Name: Rejie
- Processor: Intel (R) Core (TM) i3-1005G1 CPU @ 1.20GHz 1.19 GHz 
- Installed Ram: 8.00GB
- Device ID: DCC8A171-8140-468A-A47C-3A25AF3FED6
- Product ID: 00327-36251-49770-AAOEM 
- System Type: 64-bit operating System x64-based processor
- Operating System: Windows 11 Home Single Language version 23H2 

Software

- Microsoft Word 
- Visual Studio 
- Draw IO 
- MS SQL Server 2022 
- .Net Framework 4.8 
- SQL Server Management Studio 

Summary

This thesis examines the daily challenges faced by small retail business named Pamela Mabulay store in managing their inventory and sale processes, particularly focusing in their inefficiency manual systems. The study aims to develop an Inventory Management System with Point of sale with comprehensive features like real-time inventory tracking, automated reconciliation, and advanced reporting features.

To achieve these objectives, the developers conducted in-depth face to face interview to the said retail store to gather important data and solutions to their manual operations. The system was designed using Windows Presentation Foundation (WPF) to implement user-friendly and modern user interface with the use of VB.NET programming language.

The study's findings highlight the improvements in inventory accuracy, data accessibility, and operational efficiency. The implementation of real-time tracking has reduced manual errors, provided immediate access to critical data, and empowered the store with the tools necessary to make informed business decisions swiftly. This project not only meets the immediate needs of Pamela Mabuhay Store but also sets a foundation for scalable growth and continued operational enhancements.

Recommendations

Based on this study's findings, it is recommended that the Pamela Mabulay store implement a computerized Inventory Management System (IMS) that is integrated with a Point of Sale (POS) system. This suggestion stems from inefficiencies seen I n current manual procedures, such as delays in inventory updates, human mistakes in sales reports, and difficulties in controlling stock levels. Introducing this system will enable the store to address these challenges and boost overall productivity.

The suggested system ought to incorporate functions such as automated inventory updates during the sale process to reduce mistakes and ensure up-to-date stock levels. Additionally, it should implement daily automated processes for sales and inventory reconciliation to decrease discrepancies and allow store management more time.

Furthermore, improved reporting features that offer in-depth analysis on sales patterns, inventory rotation, and product effectiveness will assist in making informed business choices. These characteristics will tackle the main problems outlined in the study, like boosting operational efficiency and minimizing manual errors, resulting in improved business performance. It is advisable to organize training sessions for employees in order to facilitate a successful transition to the new system and make the most of its advantages. Maintaining a competitive edge in this industry necessitates ongoing enhancements, and implementing this system is a definite and feasible answer that directly corresponds with the research discoveries.

Data Gathering

Conclusion

Therefore, we concluded that the daily challenges faced by small retail business named Pamela Mabulay store in managing their inventory and sale processes, particularly focusing in their inefficiency manual systems. The study aims to develop an Inventory Management System with Point of sale with comprehensive features like real-time inventory tracking, automated reconciliation, and advanced reporting features.

To achieve these objectives, the developers conducted in-depth face to face interview to the said retail store to gather important data and solutions to their manual operations. The system was designed using Windows Presentation Foundation (WPF) to implement user-friendly and modern user interface with the use of VB.NET programming language.

The study's findings highlight the improvements in inventory accuracy, data accessibility, and operational efficiency. The implementation of real-time tracking has reduced manual errors, provided immediate access to critical data, and empowered the store with the tools necessary to make informed business decisions swiftly. This project not only meets the immediate needs of Pamela Mabuhay Store but also sets a foundation for scalable growth and continued operational enhancements.

Sample Receipt

Pamela Mabulay Footwear Retail Store
#725 Quezon Blvd, Zone 030 Brgy. 308
Quiapo
Manila, Philippines
09947294323
pamelamabulayfootwear@gmail.com

Date: 2024-11-23 01:24 PM

Cashier Name: xmont

RECEIPT

Product Name	Price	Quantity	Subtotal
Shoesfour	222.00	2	444.00
Shoesthree	2,000.00	2	4,000.00
Shoesone	5,000.00	1	5,000.00
sdfs	3,434.00	1	3,434.00

Bill:13000 **Change: 122**

Total: 12,878.00

Pamela Mabulay Footwear Retail Store
#725 Quezon Blvd, Zone 030 Brgy. 308 Quiapo
Manila, Philippines

Date: 2024-11-23

CashierID	Username	Firstname	Lastname	Email	CreatedAt
23	Reyven	Reyven	Eran	reyven@gmail.com	2024-09-28 05:44
24	Ryan	Ryan	Escalante	ryanescalante@gmail.com	2024-10-09 01:05
25	xmont	Jerald	Montemor	xmontemor@gmail.com	2024-10-12 07:49
28	xmonts	Xmonts	Montemor	xmontemorjerald@gmail.com	2024-11-01 11:47
37	Shin	Celestino	Flores	shinchanflores@gmail.com	2024-11-04 08:48

Pamela Mabulay Footwear Retail Store
#725 Quezon Blvd, Zone 030 Brgy. 308 Quiapo
Manila, Philippines

Date: 2024-11-23

ProductID	ProductName	Current Stock	Original Stock	LastUpdated
1065	Shoesone	419	434	2024-10-21 12:07
1066	Shoestwo	0	1	2024-10-21 12:09
1067	Shoesthree	3488	3499	2024-10-21 12:36
1068	Shoesfour	3423	3434	2024-10-25 07:10
1069	Shoesfive	1	1	2024-10-25 11:35
1070	Shoessix	2099	2131	2024-10-26 01:32
1072	dsfs	1	1	2024-10-30 09:18
1073	sdfs	225	228	2024-10-30 11:21
1074	new	596	780	2024-11-04 09:46
2074	Hey	1	1	2024-11-13 01:44
2075	resrs	1	3	2024-11-16 04:25

Pamela Mabulay Footwear Retail Store
#725 Quezon Blvd, Zone 030 Brgy. 308 Quiapo
Manila, Philippines

Date: 2024-11-26

SaleID	CashierID	Name	Price	Qty	Subtotal	Date	TotalAmount
1073	25	test	$121.00	1	$121.00	2024-10-21 09:07	$121.00
1074	25	test	$121.00	1	$121.00	2024-10-21 09:10	$121.00
1075	25	test	$121.00	1	$121.00	2024-10-21 09:14	$121.00
1076	25	test	$121.00	1	$121.00	2024-10-21 09:16	$121.00
1077	25	test	$121.00	1	$121.00	2024-10-21 09:17	$121.00
1078	25	ok	$2.00	30	$60.00	2024-10-21 12:40	$60.00
1079	25	sdfsf	$323.00	1	$323.00	2024-10-21 12:42	
1079	25	dfgd	$32.00	2	$64.00	2024-10-21 12:42	
1079	25	test	$3,434.00	3	$10,302.00	2024-10-21 12:42	
1079	25	ok	$2.00	1	$2.00	2024-10-21 12:42	$10,691.00
1080	25	dfgd	$32.00	6	$192.00	2024-10-21 13:52	
1080	25	sdfsf	$323.00	1	$323.00	2024-10-21 13:52	$515.00

Grand Total: $11,871.00

Source Code

Login

```
Private Sub Login()
    Dim connections As New ConnectionString()
    Dim usernames As String = Username.Text
    Dim passwords As String = If(revealmode.IsChecked, PasswordTextBox.Text, Password.Password)

    If String.IsNullOrWhiteSpace(usernames) OrElse String.IsNullOrWhiteSpace(passwords) Then
        MessageBox.Show("Username and password are required.", "Validation Error", MessageBoxButton.OK, MessageBoxImage.Error)
        Return
    End If
    Dim adminQuery As String = "SELECT PasswordHash, Salt FROM Admin WHERE Username = @Username COLLATE Latin1_General_BIN"
    Dim cashierQuery As String = "SELECT PasswordHash, Salt FROM Cashier WHERE Username = @Username COLLATE Latin1_General_BIN"
    Using connection As New SqlConnection(connections.connectionString)
        connection.Open()
        Using adminCommand As New SqlCommand(adminQuery, connection)
            adminCommand.Parameters.AddWithValue("@Username", usernames)

            Using adminReader As SqlDataReader = adminCommand.ExecuteReader()
                If adminReader.Read() Then
                    Dim storedPasswordHash As Byte() = CType(adminReader("PasswordHash"), Byte())
                    Dim storedSalt As Byte() = CType(adminReader("Salt"), Byte())
```

```
                Dim hashedPassword As Byte() = HashPassword(passwords, storedSalt)
                If storedPasswordHash.SequenceEqual(hashedPassword) Then
                    Dim adminDash As New MainWindow()
                    adminDash.Show()
                    Clear()
                    Me.Close()
                    Return
                Else
                    MessageBox.Show("Invalid username or password.", "Login Failed",
MessageBoxButton.OK, MessageBoxImage.Error)
                    Clear()
                    Return
                End If
            End If
        End Using
    End Using

    Using cashierCommand As New SqlCommand(cashierQuery, connection)
        cashierCommand.Parameters.AddWithValue("@Username", usernames)

        Using cashierReader As SqlDataReader = cashierCommand.ExecuteReader()
            If cashierReader.Read() Then
                Dim storedPasswordHash As Byte() = CType(cashierReader("PasswordHash"),
Byte())
                Dim storedSalt As Byte() = CType(cashierReader("Salt"), Byte())

                Dim hashedPassword As Byte() = HashPassword(passwords, storedSalt)

                If storedPasswordHash.SequenceEqual(hashedPassword) Then
```

```
                    Dim main As New POS(usernames)
                    main.Show()
                    Me.Close()
                    Clear()
                Else
                    MessageBox.Show("Invalid username or password.", "Login Failed",
MessageBoxButton.OK, MessageBoxImage.Error)
                    Clear()
                End If
            Else
                MessageBox.Show("Invalid username or password.", "Login Failed",
MessageBoxButton.OK, MessageBoxImage.Error)
                Clear()
            End If
        End Using
    End Using

End Using
End Sub
```

FORGOT PASSWORD

```
Private Sub Verify_Click(sender As Object, e As RoutedEventArgs)
    Dim emailInput As String = Email.Text
    If String.IsNullOrWhiteSpace(emailInput) Then
        MessageBox.Show("Please enter an email address.", "Validation Error",
MessageBoxButton.OK, MessageBoxImage.Error)
        Return
    End If
    Dim connections As New ConnectionString()
    Dim query As String = "SELECT Email FROM Admin WHERE Email = @Email UNION
SELECT Email FROM Cashier WHERE Email = @Email"
    Dim foundEmail As String = String.Empty
    Using connection As New SqlConnection(connections.connectionString)
        Using command As New SqlCommand(query, connection)
            command.Parameters.AddWithValue("@Email", emailInput)
            connection.Open()
            Using reader As SqlDataReader = command.ExecuteReader()
                If reader.Read() Then
                    foundEmail = reader("Email").ToString()
                End If
            End Using
        End Using
    End Using
    If Not String.IsNullOrEmpty(foundEmail) Then
        Dim otp As String = GenerateOTP()

        SendOTPEmail(foundEmail, otp)
        Application.Current.Properties("OTP") = otp
```

```
            Application.Current.Properties("Email") = foundEmail
            Code.IsReadOnly = False
        Else
  MessageBox.Show("Email not found!", "Error", MessageBoxButton.OK,
MessageBoxImage.Error)
            Email.Text = ""

        End If
    End Sub

Public Sub SendOTPEmail(emails As String, otp As String)
        Dim mail As New MailMessage()
        Dim smtpServer As New SmtpClient("smtp.gmail.com")

        mail.From = New MailAddress("xmontemorjerald@gmail.com")
        mail.To.Add(emails)
        mail.Subject = "Your OTP Code"
        mail.Body = "Your OTP is: " & otp

        smtpServer.Port = 587
        smtpServer.Credentials = New Net.NetworkCredential("xmontemorjerald@gmail.com",
"wkwp iyfm umur lhse")
        smtpServer.EnableSsl = True

        Try
            smtpServer.Send(mail)
            MessageBox.Show("OTP sent successfully!", "Success", MessageBoxButton.OK,
MessageBoxImage.Information)
        Catch ex As Exception
```

```
        MessageBox.Show("Error sending OTP: " & ex.Message, "Error", MessageBoxButton.OK, MessageBoxImage.Error)
        Email.Text = ""

    End Try
  End Sub

Private Sub OTP_Click(sender As Object, e As RoutedEventArgs)
    Dim enteredOtp As String = Code.Text
    Dim storedOtp As String = TryCast(Application.Current.Properties("OTP"), String)

    If String.IsNullOrEmpty(storedOtp) Then
        MessageBox.Show("No OTP generated. Please request a new one.", "Error", MessageBoxButton.OK, MessageBoxImage.Error)
        Email.Text = ""
        Code.Text = ""

        Return
    End If

    If enteredOtp = storedOtp Then
        MessageBox.Show("OTP verified successfully!", "Success", MessageBoxButton.OK, MessageBoxImage.Information)
        Dim emails As String = Email.Text
        Dim change As New ChangePassword(emails)
        change.Show()
        Me.Close()
        Email.Text = ""

        Clear()
```

```
        Else
            MessageBox.Show("Invalid OTP. Please try again.", "Error", MessageBoxButton.OK,
MessageBoxImage.Error)
            Code.Text = ""
        End If
    End Sub
```

CHANGE PASSWORD

```
Public Function VerifyEmailInAdmin(email As String) As Boolean
    Dim query As String = "SELECT COUNT(*) FROM Admin WHERE Email = @Email"

    Using connection As New SqlConnection(cons.connectionString)
        Using command As New SqlCommand(query, connection)
            command.Parameters.Add(New SqlParameter("@Email", SqlDbType.NVarChar,
100)).Value = email
            Try
                connection.Open()
                Dim count As Integer = Convert.ToInt32(command.ExecuteScalar())
                Return count > 0
            Catch ex As Exception
                MessageBox.Show("An error occurred: " & ex.Message, "Error",
MessageBoxButton.OK, MessageBoxImage.Error)
                Return False
            End Try
        End Using
    End Using
End Function

Public Function VerifyEmailInCashier(email As String) As Boolean
    Dim query As String = "SELECT COUNT(*) FROM Cashier WHERE Email = @Email"

    Using connection As New SqlConnection(cons.connectionString)
        Using command As New SqlCommand(query, connection)
            command.Parameters.Add(New SqlParameter("@Email", SqlDbType.NVarChar,
100)).Value = email
            Try
```

```
                connection.Open()
                Dim count As Integer = Convert.ToInt32(command.ExecuteScalar())
                Return count > 0
            Catch ex As Exception
                MessageBox.Show("An error occurred: " & ex.Message, "Error",
MessageBoxButton.OK, MessageBoxImage.Error)
                NewPassword.Password = ""
                ConfirmPassword.Password = ""
                Return False
            End Try
        End Using
    End Using
End Function

Public Sub UpdatePasswordInTable(tableName As String, newPassword As String)
    Dim saltBytes As Byte() = GenerateSalt(16)
    Dim passwordHash As Byte() = HashPassword(newPassword, saltBytes)

    Dim query As String = $"UPDATE [dbo].[{tableName}] SET [PasswordHash] =
@PasswordHash, [Salt] = @Salt WHERE [Email] = @Email"

    Using connection As New SqlConnection(cons.connectionString)
        Using command As New SqlCommand(query, connection)
            command.Parameters.Add(New SqlParameter("@PasswordHash",
SqlDbType.VarBinary, passwordHash.Length)).Value = passwordHash
            command.Parameters.Add(New SqlParameter("@Salt", SqlDbType.VarBinary,
saltBytes.Length)).Value = saltBytes
            command.Parameters.Add(New SqlParameter("@Email", SqlDbType.NVarChar,
100)).Value = Email
```

```
            Try
                connection.Open()
                Dim rowsAffected As Integer = command.ExecuteNonQuery()

                If rowsAffected > 0 Then
                    MessageBox.Show("Password updated successfully!", "Success",
MessageBoxButton.OK, MessageBoxImage.Information)
                    Dim login As New LoginView()
                    login.Show()
                    Me.Close()
                Else
                    MessageBox.Show("Failed to update the password.", "Error",
MessageBoxButton.OK, MessageBoxImage.Error)

                End If
            Catch ex As Exception
                MessageBox.Show("An error occurred: " & ex.Message, "Error",
MessageBoxButton.OK, MessageBoxImage.Error)
            End Try
        End Using
    End Using
End Sub
```

DASHBOARD

```
Private Sub FetchMonthlySalesData()
    Dim monthlySales As New Dictionary(Of String, Decimal)

    Dim query As String = "SELECT YEAR(SaleDate) AS Year, MONTH(SaleDate) AS Month, SUM(TotalAmount) AS MonthlyTotal " &
                          "FROM [Pamela].[dbo].[Sales] " &
                          "GROUP BY YEAR(SaleDate), MONTH(SaleDate) " &
                          "ORDER BY YEAR(SaleDate), MONTH(SaleDate);"

    Using connection As New SqlConnection(cons.connectionString)
        Dim command As New SqlCommand(query, connection)
        Dim adapter As New SqlDataAdapter(command)
        Dim dataTable As New DataTable()

        connection.Open()
        adapter.Fill(dataTable)

        For Each row As DataRow In dataTable.Rows
            Dim year As Integer = Convert.ToInt32(row("Year"))
            Dim month As Integer = Convert.ToInt32(row("Month"))
            Dim totalAmount As Decimal = Convert.ToDecimal(row("MonthlyTotal"))

            Dim monthYear As String = $"{year}-{month:D2}" ' Format as Year-Month (e.g., "2024-01")
            monthlySales(monthYear) = totalAmount
        Next
    End Using
```

```
    ' Now that we have the monthly sales data, let's populate the chart
    PopulateChart(monthlySales)
End Sub

Private Sub PopulateChart(monthlySales As Dictionary(Of String, Decimal))
    ' Create a list to hold the values for the chart
    Dim salesValues2024 As New ChartValues(Of Decimal)
    Dim salesValues2023 As New ChartValues(Of Decimal)
    Dim labels As New List(Of String)

    For Each monthYear In monthlySales.Keys
        If monthYear.StartsWith("2024") Then
            salesValues2024.Add(monthlySales(monthYear))
        ElseIf monthYear.StartsWith("2023") Then
            salesValues2023.Add(monthlySales(monthYear))
        End If

        labels.Add(monthYear.Substring(5)) ' Extract the month part for the labels
    Next

    ' Assuming you have a reference to the chart
    Dim cartesianChart = CType(FindName("test"), CartesianChart) ' Change "yourChartName" to the actual name of your chart

    ' Update the series with new data
    cartesianChart.Series.Add(New ColumnSeries With {
        .Title = "2024",
        .Values = salesValues2024,
```

```
        .Fill = Brushes.Blue ' Add color for the series
    })
    cartesianChart.Series.Add(New ColumnSeries With {
    .Title = "2023",
    .Values = salesValues2023,
    .Fill = Brushes.Red ' Add color for the series
})


    ' Update the X-axis labels
    cartesianChart.AxisX(0).Labels = labels
End Sub

Private Sub ProductCount()
    Dim query As String = "SELECT COUNT(*) FROM Product"
    Using connection As New SqlConnection(cons.connectionString)
        connection.Open()
        Using cmd As New SqlCommand(query, connection)

            Dim count As Integer = Convert.ToInt32(cmd.ExecuteScalar())
            ProductCounts.Text = count
        End Using

    End Using

End Sub

Private Sub SaleCount()
    Dim query As String = "SELECT COUNT(*) FROM Sales"
```

```
    Using connection As New SqlConnection(cons.connectionString)
        connection.Open()
        Using cmd As New SqlCommand(query, connection)

            Dim count As Integer = Convert.ToInt32(cmd.ExecuteScalar())
            Sale.Text = count
        End Using

    End Using

End Sub


Private Sub CashierCount()
    Dim query As String = "SELECT COUNT(*) FROM Cashier"
    Using connection As New SqlConnection(cons.connectionString)
        connection.Open()
        Using cmd As New SqlCommand(query, connection)

            Dim count As Integer = Convert.ToInt32(cmd.ExecuteScalar())
            CashierCounts.Text = count
        End Using

    End Using

End Sub

Private Sub SalesCount()
    Dim query As String = "SELECT COUNT(*) FROM Sale"
```

```
    Using connection As New SqlConnection(cons.connectionString)
        connection.Open()
        Using cmd As New SqlCommand(query, connection)

            Dim count As Integer = Convert.ToInt32(cmd.ExecuteScalar())
            CashierCounts.Text = count
        End Using

    End Using

End Sub

Private Sub InventoryCount()
    Dim query As String = "SELECT COUNT(*) FROM Inventory WHERE Quantity = 0"
    Using connection As New SqlConnection(cons.connectionString)
        connection.Open()
        Using cmd As New SqlCommand(query, connection)

            Dim count As Integer = Convert.ToInt32(cmd.ExecuteScalar())

            If count = 0 Then
                ZeroCounts.Text = count

            Else
                ZeroCounts.Text = count
                ZeroCounts.Foreground = Brushes.Red

            End If
```

```
        End Using

    End Using

End Sub
Private Sub LowCounts()
    Dim query As String = "SELECT COUNT(*) FROM Inventory WHERE Quantity <= 5"
    Using connection As New SqlConnection(cons.connectionString)
        connection.Open()
        Using cmd As New SqlCommand(query, connection)

            Dim count As Integer = Convert.ToInt32(cmd.ExecuteScalar())

            If count = 0 Then
                Lowcount.Text = count

            Else
                Lowcount.Text = count
                Lowcount.Foreground = New
SolidColorBrush(ColorConverter.ConvertFromString("#FFD700"))

            End If

        End Using

    End Using

End Sub
```

PRODUCTS

```
Public Sub FetchProductData()
    Dim prod As New ObservableCollection(Of Product)()

    Dim query As String = "SELECT p.ProductID, p.ProductName, p.Price, p.Description, p.CategoryID, c.CategoryName, p.Brand, p.Size, p.Color, p.CreatedAt " &
                          "FROM Product p JOIN Category c ON p.CategoryID = c.CategoryID"
    Using connection As New SqlConnection(con.connectionString)
        Dim command As New SqlCommand(query, connection)
        Dim adapter As New SqlDataAdapter(command)
        Dim dataTable As New DataTable()

        connection.Open()
        adapter.Fill(dataTable)

        For Each row As DataRow In dataTable.Rows
            prod.Add(New Product With {
            .ProductID = Convert.ToInt32(row("ProductID")),
            .ProductName = row("ProductName").ToString(),
            .Price = Convert.ToDecimal(row("Price")),
            .Description = row("Description"),
            .CategoryID = Convert.ToInt32(row("CategoryID")),
             .CategoryName = row("CategoryName").ToString(),
            .Brand = row("Brand").ToString(),
            .Size = row("Size").ToString(),
            .Color = row("Color").ToString(),
            .CreatedAt = CDate(row("CreatedAt")).ToString("yyyy-MM-dd hh:mm")
        })
```

```
            Next

            productDataGrid.ItemsSource = prod
        End Using
    End Sub

Private Sub FilterProductTable()
        Dim input As String = Search.Text.Trim()

        If String.IsNullOrEmpty(input) Then
            MessageBox.Show("Please provide a value.", "Error", MessageBoxButton.OK,
MessageBoxImage.Error)
            Return
        End If

        Dim products As New List(Of Product)()
        Dim isNumericInput As Boolean = Decimal.TryParse(input, New Decimal())
        Dim isIntegerInput As Boolean = Integer.TryParse(input, New Integer())
        Dim isDateInput As Boolean = DateTime.TryParse(input, Nothing) ' Check if input can be
parsed as Date

        ' Prepare SQL query with conditional parameters based on input type
        Dim query As New StringBuilder()
        query.Append("SELECT p.*, c.CategoryName FROM PRODUCT p ")
        query.Append("INNER JOIN Category c ON p.CategoryID = c.CategoryID ")
        query.Append("WHERE 1=1 ") ' Always true, simplifies adding conditions

        ' Conditions based on user input
        If Not String.IsNullOrEmpty(input) Then
```

```
    query.Append("AND (p.ProductName LIKE '%' + @ProductName + '%' ")
    query.Append("OR p.Description LIKE '%' + @Description + '%' ")
    query.Append("OR p.Brand LIKE '%' + @Brand + '%' ")
    query.Append("OR p.Size LIKE '%' + @Size + '%' ")
    query.Append("OR p.Color LIKE '%' + @Color + '%' ")

    If isNumericInput Then
        query.Append("OR p.Price = @Price ")
    End If

    If isIntegerInput Then
        query.Append("OR p.CategoryID = @CategoryID ")
        query.Append("OR p.ProductID = @ProductID ")
    End If

    If isDateInput Then
        query.Append("OR CONVERT(DATE, p.CreatedAt) = @CreatedAt ")
    End If

    query.Append("OR c.CategoryName LIKE '%' + @CategoryName + '%' )")
End If

Using connection As New SqlConnection(con.connectionString)
    connection.Open()

    Using cmd As New SqlCommand(query.ToString(), connection)
        cmd.Parameters.AddWithValue("@ProductName", input)
        cmd.Parameters.AddWithValue("@Description", input)
        cmd.Parameters.AddWithValue("@Brand", input)
```

```
cmd.Parameters.AddWithValue("@Size", input)
cmd.Parameters.AddWithValue("@Color", input)
cmd.Parameters.AddWithValue("@CategoryName", input)

If isNumericInput Then
   cmd.Parameters.AddWithValue("@Price", Convert.ToDecimal(input))
End If
If isIntegerInput Then
   cmd.Parameters.AddWithValue("@CategoryID", Convert.ToInt32(input))
   cmd.Parameters.AddWithValue("@ProductID", Convert.ToInt32(input))
End If

If isDateInput Then
   Dim parsedDate As DateTime
   parsedDate = DateTime.Parse(input) ' Parsing the input date
   cmd.Parameters.AddWithValue("@CreatedAt", parsedDate)
Else
   cmd.Parameters.Add("@CreatedAt", SqlDbType.Date).Value = DBNull.Value
End If

Using reader As SqlDataReader = cmd.ExecuteReader()
   While reader.Read()
      Dim filter As New Product() With {
      .ProductID = Convert.ToInt32(reader("ProductID")),
      .ProductName = reader("ProductName").ToString(),
      .Description = reader("Description").ToString(),
      .Brand = reader("Brand").ToString(),
      .Size = reader("Size").ToString(),
      .Color = reader("Color").ToString(),
```

```
                .Price = Convert.ToDecimal(reader("Price")),
                .CategoryID = Convert.ToInt32(reader("CategoryID")),
                .CategoryName = reader("CategoryName").ToString(),
                .CreatedAt = Convert.ToDateTime(reader("CreatedAt")).ToString("yyyy-MM-dd
hh:mm")
            }

                products.Add(filter)
            End While
        End Using
    End Using
End Using

productDataGrid.ItemsSource = products
End Sub

Private Sub PrintToPDF(dataGrid As DataGrid, filePath As String)
    Using pdfWriter As New PdfWriter(filePath)
        Using pdfDocument As New PdfDocument(pdfWriter)
            Dim document As New Document(pdfDocument)

            ' Add Logo
            Dim logoPath As String = "D:\VB_THESIS_WPS\Images\logo.png"
            Dim logo As Image = New Image(ImageDataFactory.Create(logoPath))
            logo.SetHorizontalAlignment(HorizontalAlignment.Center)
            logo.SetWidth(150)
            document.Add(logo)

            ' Address paragraph
```

```
        Dim address As String = "Pamela Mabulay Footwear Retail Store" & vbCrLf &
                                "#725 Quezon Blvd, Zone 030 Brgy. 308 Quiapo" & vbCrLf &
                                "Manila, Philippines"
        Dim addressParagraph As New Paragraph(address)
        addressParagraph.SetTextAlignment(TextAlignment.CENTER)
        addressParagraph.SetFontSize(12)
        document.Add(addressParagraph)
        document.Add(New Paragraph().SetHeight(20))

        'Printed date paragraph
        Dim printedDate As String = DateTime.Today.ToString("yyyy-MM-dd",
CultureInfo.InvariantCulture)
        Dim concatDate As String = $"Date: {printedDate}"
        Dim printedDateParagraph As New Paragraph(concatDate)
        printedDateParagraph.SetTextAlignment(TextAlignment.LEFT)
        printedDateParagraph.SetFontSize(12)
        document.Add(printedDateParagraph)
        document.Add(New Paragraph().SetHeight(10))


        Dim table As New Table(dataGrid.Columns.Count - 1)
        table.SetWidth(UnitValue.CreatePercentValue(100))

        For Each column As DataGridColumn In dataGrid.Columns
            If Not TypeOf column Is DataGridTemplateColumn Then
                table.AddHeaderCell(New Cell().Add(New Paragraph(column.Header.ToString())))
            End If
        Next
```

```
            For Each item In dataGrid.ItemsSource
                Dim product As Product = CType(item, Product)
                table.AddCell(New Cell().Add(New Paragraph(product.ProductID.ToString())))
                table.AddCell(New Cell().Add(New Paragraph(product.ProductName.ToString())))
                table.AddCell(New Cell().Add(New Paragraph(product.Description.ToString())))
                table.AddCell(New Cell().Add(New Paragraph(product.CategoryName.ToString())))
                table.AddCell(New Cell().Add(New Paragraph(product.Brand.ToString())))
                table.AddCell(New Cell().Add(New Paragraph(product.Size.ToString())))
                table.AddCell(New Cell().Add(New Paragraph(product.Color.ToString())))
                table.AddCell(New Cell().Add(New Paragraph(product.Price.ToString())))

                table.AddCell(New Cell().Add(New
Paragraph(String.Format(CultureInfo.InvariantCulture, "{0:yyyy-MM-dd}", product.CreatedAt))))
            Next

            document.Add(table)
            document.Close()
            MessageBox.Show("PDF created successfully: ", "Success", MessageBoxButton.OK,
MessageBoxImage.Information)
        End Using
    End Using

    Process.Start(New ProcessStartInfo(filePath) With {
    .UseShellExecute = True
})
End Sub
```

INVENTORY

```
Private Sub FilterInventoryTable()
    Dim input As String = Search.Text.Trim()

    If String.IsNullOrEmpty(input) Then
        MessageBox.Show("Error", "Please provide a value", MessageBoxButton.OK, MessageBoxImage.Error)
        Return
    End If

    Dim inventoryItems As New List(Of ProductInventory)()

    Dim q As String = "SELECT P.InventoryID, P.ProductID, S.ProductName, P.Quantity, P.OriginalStock, P.LastUpdated " &
                      "FROM Inventory P INNER JOIN Product S ON P.ProductID = S.ProductID " &
                      "WHERE (@Search IS NULL OR " &
                      "CAST(P.InventoryID AS NVARCHAR) LIKE '%' + @Search + '%' OR " &
                      "CAST(P.ProductID AS NVARCHAR) LIKE '%' + @Search + '%' OR " &
                      "CAST(P.Quantity AS NVARCHAR) LIKE '%' + @Search + '%' OR " &
                      "CAST(P.OriginalStock AS NVARCHAR) LIKE '%' + @Search + '%' OR " &
                      "CONVERT(NVARCHAR, P.LastUpdated, 120) LIKE '%' + @Search + '%' OR " &
                      "S.ProductName LIKE '%' + @Search + '%')"

    Using connection As New SqlConnection(con.connectionString)
        connection.Open()

        Using cmd As New SqlCommand(q, connection)
            cmd.Parameters.AddWithValue("@Search", If(String.IsNullOrEmpty(input), DBNull.Value, input))
```

```
            Using reader As SqlDataReader = cmd.ExecuteReader()
               While reader.Read()
                  Dim inventoryItem As New ProductInventory() With {
               .InventoryID = Convert.ToInt32(reader("InventoryID")),
               .ProductID = Convert.ToInt32(reader("ProductID")),
               .ProductName = reader("ProductName").ToString(),
               .LastUpdated = Convert.ToDateTime(reader("LastUpdated")).ToString("yyyy-MM-dd
hh:mm"),
               .CurrentStock = Convert.ToInt32(reader("Quantity")),
               .OriginalStock = Convert.ToInt32(reader("OriginalStock"))
            }
                  inventoryItems.Add(inventoryItem)
               End While
            End Using
         End Using
      End Using

      productDataGrid.ItemsSource = inventoryItems
   End Sub


   Public Sub FetchProductData()

      Dim prod As New ObservableCollection(Of ProductInventory)()

      Dim query As String = "SELECT P.InventoryID, P.ProductID, S.ProductName, P.Quantity,
P.OriginalStock, P.LastUpdated
                  FROM Inventory P
```

```
                INNER JOIN Product S ON P.ProductID = S.ProductID"
    Using connection As New SqlConnection(con.connectionString)
        Dim command As New SqlCommand(query, connection)
        Dim adapter As New SqlDataAdapter(command)
        Dim dataTable As New DataTable()

        connection.Open()
        adapter.Fill(dataTable)

        For Each row As DataRow In dataTable.Rows
            ' Create the product inventory object
            Dim inventoryItem As New ProductInventory With {
            .InventoryID = Convert.ToInt32(row("InventoryID")),
            .ProductID = Convert.ToInt32(row("ProductID")),
            .ProductName = row("ProductName").ToString(),
            .CurrentStock = Convert.ToInt32(row("Quantity")),
            .OriginalStock = Convert.ToInt32(row("OriginalStock")),
            .LastUpdated = CDate(row("LastUpdated")).ToString("yyyy-MM-dd hh:mm")
        }


            prod.Add(inventoryItem)
        Next

        productDataGrid.ItemsSource = prod
    End Using
End Sub

Private Sub Button_Click(sender As Object, e As RoutedEventArgs)
```

```
        Dim deleteButton As Button = CType(sender, Button)
        Dim selectedProduct As ProductInventory = CType(deleteButton.DataContext, ProductInventory)
        Dim productID As String = selectedProduct.ProductID

        Dim delete As MsgBoxResult = MessageBox.Show("Are you sure you want to delete this product record?", "Confirmation", MessageBoxButton.YesNo, MessageBoxImage.Question)

        If delete = MsgBoxResult.Yes Then
            Dim deleteInventoryQuery As String = "DELETE FROM Inventory WHERE ProductID = @ProductID"
            Dim deleteProductQuery As String = "DELETE FROM Product WHERE ProductID = @ProductID"

            Try
                Using connection As New SqlConnection(con.connectionString)
                    connection.Open()
                    Using transaction As SqlTransaction = connection.BeginTransaction()
                        Try
                            Using cmd As New SqlCommand(deleteInventoryQuery, connection, transaction)
                                cmd.Parameters.AddWithValue("@ProductID", productID)
                                cmd.ExecuteNonQuery()
                            End Using

                            Using cmd As New SqlCommand(deleteProductQuery, connection, transaction)
                                cmd.Parameters.AddWithValue("@ProductID", productID)
                                cmd.ExecuteNonQuery()
                            End Using
```

```
                    transaction.Commit()
                    MessageBox.Show("Record deleted successfully.", "Success",
MessageBoxButton.OK, MessageBoxImage.Information)

                    FetchProductData()
                Catch ex As Exception
                    transaction.Rollback()
                    MessageBox.Show("An error occurred: " & ex.Message, "Error",
MessageBoxButton.OK, MessageBoxImage.Error)
                End Try
            End Using
        End Using
    Catch ex As Exception
        MessageBox.Show("An error occurred while connecting to the database: " & ex.Message,
"Error", MessageBoxButton.OK, MessageBoxImage.Error)
    End Try
  End If
End Sub



Private Sub PrintToPDF(dataGrid As DataGrid, filePath As String)
    Using pdfWriter As New PdfWriter(filePath)
        Using pdfDocument As New PdfDocument(pdfWriter)
            Dim document As New Document(pdfDocument)

            ' Add Logo
            Dim logoPath As String = "D:\VB_THESIS_WPS\Images\logo.png"
            Dim logo As Image = New Image(ImageDataFactory.Create(logoPath))
            logo.SetHorizontalAlignment(HorizontalAlignment.Center)
```

```
        logo.SetWidth(150)
        document.Add(logo)

        ' Address paragraph
        Dim address As String = "Pamela Mabulay Footwear Retail Store" & vbCrLf &
                                "#725 Quezon Blvd, Zone 030 Brgy. 308 Quiapo" & vbCrLf &
                                "Manila, Philippines"
        Dim addressParagraph As New Paragraph(address)
        addressParagraph.SetTextAlignment(TextAlignment.CENTER)
        addressParagraph.SetFontSize(12)
        document.Add(addressParagraph)
        document.Add(New Paragraph().SetHeight(20))



        'Printed date paragraph
        Dim printedDate As String = DateTime.Today.ToString("yyyy-MM-dd",
CultureInfo.InvariantCulture)
        Dim concatDate As String = $"Date: {printedDate}"
        Dim printedDateParagraph As New Paragraph(concatDate)
        printedDateParagraph.SetTextAlignment(TextAlignment.LEFT)
        printedDateParagraph.SetFontSize(12)
        document.Add(printedDateParagraph)
        document.Add(New Paragraph().SetHeight(10))



        Dim table As New Table(dataGrid.Columns.Count - 1)
        table.SetWidth(UnitValue.CreatePercentValue(100))
```

```
            For Each column As DataGridColumn In dataGrid.Columns
                If Not TypeOf column Is DataGridTemplateColumn Then
                    table.AddHeaderCell(New Cell().Add(New Paragraph(column.Header.ToString())))
                End If
            Next

            For Each item In dataGrid.ItemsSource
                Dim product As ProductInventory = CType(item, ProductInventory)
                table.AddCell(New Cell().Add(New Paragraph(product.ProductID.ToString())))
                table.AddCell(New Cell().Add(New Paragraph(product.ProductName.ToString())))
                table.AddCell(New Cell().Add(New Paragraph(product.CurrentStock.ToString())))
                table.AddCell(New Cell().Add(New Paragraph(product.OriginalStock.ToString())))
                table.AddCell(New Cell().Add(New
Paragraph(String.Format(CultureInfo.InvariantCulture, "{0:yyyy-MM-dd}",
product.LastUpdated))))
            Next

            document.Add(table)
            document.Close()
            MessageBox.Show("PDF created successfully: ", "Success", MessageBoxButton.OK,
MessageBoxImage.Information)
        End Using
    End Using

    Process.Start(New ProcessStartInfo(filePath) With {
    .UseShellExecute = True
})
End Sub

Private Sub Print_Click_1(sender As Object, e As RoutedEventArgs)
```

```
        Dim rand As New Random
        Dim randomNumber As Integer = rand.Next()
        Dim randomInRange As Integer = rand.Next(1, 1000)

        Dim filePath As String = $"D:\VB_THESIS_WPS\Prints\Inventory_{randomInRange}.pdf"

        If productDataGrid.ItemsSource Is Nothing OrElse productDataGrid.Items.Count = 0 OrElse productDataGrid.Columns.Count = 0 Then
            MessageBox.Show("No data available to print.", "Print Cancelled", MessageBoxButton.OK, MessageBoxImage.Error)
            Return
        End If

        PrintToPDF(productDataGrid, filePath)
    End Sub

    Private Sub editbtn_Click(sender As Object, e As RoutedEventArgs)
        Dim edit_Button As Button = CType(sender, Button)

        Dim selectedId As ProductInventory = CType(edit_Button.DataContext, ProductInventory)

        Dim passToEdit As New StockIn(
         selectedId.InventoryID,
         Me
     )

        passToEdit.Show()
    End Sub
```

SALES

```
Private Sub FetchSale()
    Dim salesDetailsCollection As New ObservableCollection(Of SaleDetails)()

    Dim query As String = "SELECT s.[SalesID], s.[CashierID], s.[SaleDate], s.[TotalAmount], "
&
            "sd.[SalesDetailID], sd.[ProductName], sd.[Quantity], sd.[UnitPrice], sd.[TotalPrice]
" &
            "FROM [Pamela].[dbo].[Sales] s " &
            "INNER JOIN [Pamela].[dbo].[SalesDetails] sd " &
            "ON s.[SalesID] = sd.[SalesID] " &
            "ORDER BY s.[SalesID];"

    Using connection As New SqlConnection(con.connectionString)
        Dim command As New SqlCommand(query, connection)
        Dim adapter As New SqlDataAdapter(command)
        Dim dataTable As New DataTable()

        connection.Open()
        adapter.Fill(dataTable)

        For Each row As DataRow In dataTable.Rows
            Dim saleDetails As New SaleDetails With {
            .SalesDetailsID = Convert.ToInt32(row("SalesDetailID")),
            .SaleID = Convert.ToInt32(row("SalesID")),
            .CashierID = Convert.ToInt32(row("CashierID")),
            .SaleDate = If(IsDBNull(row("SaleDate")), String.Empty,
Convert.ToDateTime(row("SaleDate")).ToString("yyyy-MM-dd HH:mm")),
            .TotalAmount = Convert.ToDecimal(row("TotalAmount")),
```

```
            .ProductName = row("ProductName").ToString(),
            .Quantity = Convert.ToInt32(row("Quantity")),
            .UnitPrice = Convert.ToDecimal(row("UnitPrice")),
            .TotalPrice = Convert.ToDecimal(row("TotalPrice"))
        }

            salesDetailsCollection.Add(saleDetails)
        Next

        salesDataGrid.ItemsSource = salesDetailsCollection
    End Using
End Sub



Private Sub print_Click(sender As Object, e As RoutedEventArgs)

    Dim rand As New Random
    Dim randomNumber As Integer = rand.Next()
    Dim randomInRange As Integer = rand.Next(1, 1000)

    Dim filePath As String = $"D:\VB_THESIS_WPS\Prints\Invoice_{randomInRange}.pdf"

    PrintToPDF(salesDataGrid, filePath)
End Sub
'Pdf printer
Private Sub PrintToPDF(dataGrid As DataGrid, filePath As String)
    Using pdfWriter As New PdfWriter(filePath)
        Using pdfDocument As New PdfDocument(pdfWriter)
```

```
        Dim document As New Document(pdfDocument)

        ' Add Logo
        Dim logoPath As String = "D:\VB_THESIS_WPS\Images\logo.png" ' Change to your logo path
        Dim logo As Image = New Image(ImageDataFactory.Create(logoPath))
        logo.SetHorizontalAlignment(HorizontalAlignment.Center)
        logo.SetWidth(150)
        document.Add(logo)

        ' Add Company Address
        Dim address As String = "Pamela Mabulay Footwear Retail Store" & vbCrLf &
                                "#725 Quezon Blvd, Zone 030 Brgy. 308 Quiapo" & vbCrLf &
                                "Manila, Philippines"
        Dim addressParagraph As New Paragraph(address)
        addressParagraph.SetTextAlignment(TextAlignment.CENTER)
        addressParagraph.SetFontSize(12)
        document.Add(addressParagraph)

        document.Add(New Paragraph().SetHeight(20))

        ' Printed date paragraph
        Dim printedDate As String = DateTime.Today.ToString("yyyy-MM-dd", CultureInfo.InvariantCulture)
        Dim concatDate As String = $"Date: {printedDate}"
        Dim printedDateParagraph As New Paragraph(concatDate)
        printedDateParagraph.SetTextAlignment(TextAlignment.LEFT)
        printedDateParagraph.SetFontSize(12)
        document.Add(printedDateParagraph)
```

```
        document.Add(New Paragraph().SetHeight(10))

        ' Create a table with the number of columns in the DataGrid
        Dim table As New Table(dataGrid.Columns.Count)
        table.SetWidth(UnitValue.CreatePercentValue(100))

        ' Add headers
        For Each column As DataGridColumn In dataGrid.Columns
            If Not TypeOf column Is DataGridTemplateColumn Then ' Check if it's not an Action column
                table.AddHeaderCell(New Cell().Add(New Paragraph(column.Header.ToString())))
            End If
        Next

        Dim groupedSales As New Dictionary(Of Integer, List(Of SaleDetails))()
        Dim grandTotal As Decimal = 0 ' Variable to accumulate the grand total

        ' First pass: group items by SaleID and keep a list of associated products
        For Each item In dataGrid.ItemsSource
            Dim saleDetails As SaleDetails = CType(item, SaleDetails)

            If Not groupedSales.ContainsKey(saleDetails.SaleID) Then
                groupedSales(saleDetails.SaleID) = New List(Of SaleDetails)
            End If
            groupedSales(saleDetails.SaleID).Add(saleDetails)
        Next

        ' Second pass: add grouped items to the table, with total amount displayed only once per SaleID group
```

```
        For Each saleID In groupedSales.Keys
            Dim salesList = groupedSales(saleID)
            Dim totalAmount As Decimal = salesList.Sum(Function(sd) sd.TotalPrice)  ' Calculate total amount for this SaleID
            grandTotal += totalAmount  ' Add to grand total

            For i As Integer = 0 To salesList.Count - 1
                Dim saleDetails = salesList(i)

                ' Add SaleID, CashierID, ProductName, etc.
                table.AddCell(New Cell().Add(New Paragraph(saleDetails.SaleID.ToString())))
                table.AddCell(New Cell().Add(New Paragraph(saleDetails.CashierID.ToString())))
                table.AddCell(New Cell().Add(New Paragraph(saleDetails.ProductName)))
                table.AddCell(New Cell().Add(New Paragraph(saleDetails.UnitPrice.ToString("C", CultureInfo.CurrentCulture))))
                table.AddCell(New Cell().Add(New Paragraph(saleDetails.Quantity.ToString())))
                table.AddCell(New Cell().Add(New Paragraph(saleDetails.TotalPrice.ToString("C", CultureInfo.CurrentCulture))))
                table.AddCell(New Cell().Add(New Paragraph(String.Format(CultureInfo.InvariantCulture, "{0:yyyy-MM-dd}", saleDetails.SaleDate))))

                ' Only display the total amount in the last row of the group
                If i = salesList.Count - 1 Then
                    table.AddCell(New Cell().Add(New Paragraph(totalAmount.ToString("C", CultureInfo.CurrentCulture))))
                Else
                    table.AddCell(New Cell()) ' Empty cell for other rows in the group
                End If
            Next
        Next
```

```
            ' Add a final row for the grand total
            Dim grandTotalParagraph As New Paragraph("Grand Total: " &
grandTotal.ToString("C", CultureInfo.CurrentCulture))
            grandTotalParagraph.SetFontSize(12).SetBold() ' Optional styling



            grandTotalParagraph.SetTextAlignment(TextAlignment.RIGHT) ' Align text to the right
to fit under the column

            ' Add rows

            document.Add(table)
            document.Add(grandTotalParagraph)

            document.Close()
            MessageBox.Show("Receipt created successfully", "Success", MessageBoxButton.OK,
MessageBoxImage.Information)
        End Using
    End Using

    Process.Start(New ProcessStartInfo(filePath) With {
      .UseShellExecute = True
    })
  End Sub

  Private Sub Button_Click_1(sender As Object, e As RoutedEventArgs)

  End Sub
```

```
    Private Sub SearchSales()

        Dim searchTerm As String = Search.Text.Trim()
        Dim salesDetailsCollection As New ObservableCollection(Of SaleDetails)()

        Dim query As String = "SELECT s.[SalesID], s.[CashierID], s.[SaleDate], s.[TotalAmount], "
&
                "sd.[SalesDetailID], sd.[ProductName], sd.[Quantity], sd.[UnitPrice], sd.[TotalPrice] "
&
                "FROM [Pamela].[dbo].[Sales] s " &
                "INNER JOIN [Pamela].[dbo].[SalesDetails] sd " &
                "ON s.[SalesID] = sd.[SalesID] " &
                "WHERE sd.[ProductName] LIKE @searchTerm OR " &
                "s.[SalesID] LIKE @searchTerm OR " &
                "s.[CashierID] LIKE @searchTerm OR " &
                "CONVERT(NVARCHAR, s.[SaleDate], 120) LIKE @searchTerm OR " &
                "s.[TotalAmount] LIKE @searchTerm OR " &
                "sd.[Quantity] LIKE @searchTerm OR " &
                "sd.[UnitPrice] LIKE @searchTerm OR " &
                "sd.[TotalPrice] LIKE @searchTerm " &
                "ORDER BY s.[SalesID];"

        Using connection As New SqlConnection(con.connectionString)
            Dim command As New SqlCommand(query, connection)
            command.Parameters.AddWithValue("@searchTerm", "%" & searchTerm & "%")

            Dim adapter As New SqlDataAdapter(command)
            Dim dataTable As New DataTable()
```

```
            connection.Open()
            adapter.Fill(dataTable)

            For Each row As DataRow In dataTable.Rows
                Dim saleDetails As New SaleDetails With {
                .SalesDetailsID = Convert.ToInt32(row("SalesDetailID")),
                .SaleID = Convert.ToInt32(row("SalesID")),
                .CashierID = Convert.ToInt32(row("CashierID")),
                 .SaleDate = If(IsDBNull(row("SaleDate")), String.Empty,
Convert.ToDateTime(row("SaleDate")).ToString("yyyy-MM-dd HH:mm")),
                .TotalAmount = Convert.ToDecimal(row("TotalAmount")),
                .ProductName = row("ProductName").ToString(),
                .Quantity = Convert.ToInt32(row("Quantity")),
                .UnitPrice = Convert.ToDecimal(row("UnitPrice")),
                .TotalPrice = Convert.ToDecimal(row("TotalPrice"))
            }

                salesDetailsCollection.Add(saleDetails)
            Next

            salesDataGrid.ItemsSource = salesDetailsCollection
        End Using
    End Sub
```

ANALYTICS

```
Private Sub FetchMonthlySalesData()
    Dim monthlySales As New Dictionary(Of String, Decimal)

    Dim query As String = "SELECT YEAR(SaleDate) AS Year, MONTH(SaleDate) AS Month, SUM(TotalAmount) AS MonthlyTotal " &
                          "FROM [Pamela].[dbo].[Sales] " &
                          "GROUP BY YEAR(SaleDate), MONTH(SaleDate) " &
                          "ORDER BY YEAR(SaleDate), MONTH(SaleDate);"

    Using connection As New SqlConnection(con.connectionString)
        Dim command As New SqlCommand(query, connection)
        Dim adapter As New SqlDataAdapter(command)
        Dim dataTable As New DataTable()

        connection.Open()
        adapter.Fill(dataTable)

        For Each row As DataRow In dataTable.Rows
            Dim year As Integer = Convert.ToInt32(row("Year"))
            Dim month As Integer = Convert.ToInt32(row("Month"))
            Dim totalAmount As Decimal = Convert.ToDecimal(row("MonthlyTotal"))

            Dim monthYear As String = $"{year}-{month:D2}" ' Format as Year-Month (e.g., "2024-01")
            monthlySales(monthYear) = totalAmount
        Next
    End Using
```

```
    ' Now that we have the monthly sales data, let's populate the chart
    PopulateChart(monthlySales)
End Sub

Private Sub PopulateChart(monthlySales As Dictionary(Of String, Decimal))
    ' Create a list to hold the values for the chart
    Dim salesValues2024 As New ChartValues(Of Decimal)
    Dim salesValues2023 As New ChartValues(Of Decimal)
    Dim labels As New List(Of String)

    For Each monthYear In monthlySales.Keys
        If monthYear.StartsWith("2024") Then
            salesValues2024.Add(monthlySales(monthYear))
        ElseIf monthYear.StartsWith("2023") Then
            salesValues2023.Add(monthlySales(monthYear))
        End If

        labels.Add(monthYear.Substring(5)) ' Extract the month part for the labels
    Next

    ' Assuming you have a reference to the chart
    Dim cartesianChart = CType(FindName("test"), CartesianChart) ' Change "yourChartName" to the actual name of your chart

    ' Update the series with new data
    cartesianChart.Series.Add(New ColumnSeries With {
        .Title = "2024",
        .Values = salesValues2024,
```

```
        .Fill = Brushes.Blue ' Add color for the series
    })
    cartesianChart.Series.Add(New ColumnSeries With {
    .Title = "2023",
    .Values = salesValues2023,
    .Fill = Brushes.Red ' Add color for the series
})

    ' Update the X-axis labels
    cartesianChart.AxisX(0).Labels = labels
End Sub
```

EMPLOYEE

```
Dim converter As New BrushConverter()
    Dim myArray As String() = {"#1098AD", "#1E88E5", "#FF8F00", "#FF5252", "#6741D9",
"#0CA678"}
    Dim fletter As String
    Dim brush As Brush
    Public Sub FetchCashierData()

        Dim cashiers As New ObservableCollection(Of Cashier)()

        Dim query As String = "SELECT * FROM Cashier"

        Using connection As New SqlConnection(con.connectionString)
            Dim command As New SqlCommand(query, connection)
            Dim adapter As New SqlDataAdapter(command)
            Dim dataTable As New DataTable()

            connection.Open()
            adapter.Fill(dataTable)

            For i As Integer = 0 To dataTable.Rows.Count - 1
                Dim row As DataRow = dataTable.Rows(i)

                Dim colorString As String = myArray(i Mod myArray.Length)

                brush = CType(converter.ConvertFromString(colorString), Brush)

                Dim fname As String = row("FirstName").ToString()
```

```
            fletter = If(fname.Length > 0, fname.Substring(0, 1), String.Empty)

            cashiers.Add(New Cashier With {
                .Character = fletter,
                .BgColor = brush,
                .CashierID = row("CashierID").ToString(),
                .Username = row("Username").ToString(),
                .FirstName = row("FirstName").ToString(),
                .LastName = row("LastName").ToString(),
                .Email = row("Email").ToString(),
                .CreatedAt = CDate(row("CreatedAt")).ToString("yyyy-MM-dd hh:mm")
            })
        Next
        cashierDataGrid.ItemsSource = cashiers
    End Using
End Sub

Private Sub UserControl_Loaded(sender As Object, e As RoutedEventArgs)

End Sub

Private Sub Delete_Click(sender As Object, e As RoutedEventArgs)
    Dim deleteButton As Button = CType(sender, Button)

    Dim selectedCashier As Cashier = CType(deleteButton.DataContext, Cashier)

    Dim cashierId As String = selectedCashier.CashierID
```

```
    Dim result As MessageBoxResult = MessageBox.Show("Are you sure you want to delete this cashier?", "Confirmation", MessageBoxButton.YesNo, MessageBoxImage.Question)

    If result = MessageBoxResult.Yes Then
        Dim query As String = "DELETE FROM Cashier WHERE CashierID = @CashierID"

        Using connection As New SqlConnection(con.connectionString)
            Try
                connection.Open()
                Dim cmd As New SqlCommand(query, connection)
                cmd.Parameters.AddWithValue("@CashierID", cashierId)

                cmd.ExecuteNonQuery()
                MessageBox.Show("Cashier deleted successfully.", "Success", MessageBoxButton.OK, MessageBoxImage.Information)

                FetchCashierData()

            Catch ex As Exception
                MessageBox.Show("An error occurred: " & ex.Message, "Error", MessageBoxButton.OK, MessageBoxImage.Error)
            End Try
        End Using
    End If

  End Sub

  Private Sub Search_TextChanged(sender As Object, e As TextChangedEventArgs)

    If Search.Text.Length > 0 Then
```

```
        FilterProductTable()

    Else
        FetchCashierData()

    End If

End Sub

Private Sub Search_Click(sender As Object, e As RoutedEventArgs)
    FilterProductTable()
End Sub

Private Sub FilterProductTable()

    Dim input As String = Search.Text.Trim()
    Dim products As New List(Of Cashier)()
    Dim q As String = "SELECT * FROM Cashier WHERE " &
            "(UserName LIKE '%' + @UserName + '%' OR @UserName = '') " &
            "OR (FirstName LIKE '%' + @FirstName + '%' OR @FirstName = '') " &
            "OR (LastName LIKE '%' + @LastName + '%' OR @LastName = '') " &
            "OR (Email LIKE '%' + @Email + '%' OR @Email = '') " &
            "OR (ISNUMERIC(@CashierID) = 1 AND CashierID = @CashierID) " &
            "OR (CONVERT(DATE, CreatedAt) = @CreatedAt)"


    Using connection As New SqlConnection(con.connectionString)
        connection.Open()
```

```
        Using cmd As New SqlCommand(q, connection)
            cmd.Parameters.Add("@UserName", SqlDbType.NVarChar).Value = input
            cmd.Parameters.Add("@FirstName", SqlDbType.NVarChar).Value = input
            cmd.Parameters.Add("@LastName", SqlDbType.NVarChar).Value = input
            cmd.Parameters.Add("@Email", SqlDbType.NVarChar).Value = input
            cmd.Parameters.Add("@CashierID", SqlDbType.Int).Value = If(IsNumeric(input),
CInt(input), DBNull.Value)

            Dim parsedDate As DateTime
            cmd.Parameters.Add("@CreatedAt", SqlDbType.Date).Value =
If(DateTime.TryParse(input, parsedDate), parsedDate, DBNull.Value)


            Using reader As SqlDataReader = cmd.ExecuteReader()
                Dim i As Integer = 0
                While reader.Read()
                    Dim colorString As String = myArray(i Mod myArray.Length)
                    brush = CType(converter.ConvertFromString(colorString), Brush)

                    Dim fname As String = reader("FirstName").ToString()
                    fletter = If(fname.Length > 0, fname.Substring(0, 1), String.Empty)

                    Dim filter As New Cashier() With {
                        .CashierID = Convert.ToInt32(reader("CashierID")),
                        .Username = reader("UserName").ToString(),
                        .FirstName = reader("FirstName").ToString(),
                        .Email = reader("Email").ToString(),
                        .LastName = reader("LastName").ToString(),
                        .Character = fletter,
```

```
                .BgColor = brush,
                .CreatedAt = CDate(reader("CreatedAt")).ToString("yyyy-MM-dd hh:mm")
            }

            products.Add(filter)
            i += 1
        End While
    End Using
End Using
End Using

cashierDataGrid.ItemsSource = products
End Sub


Private Sub Button_Click_1(sender As Object, e As RoutedEventArgs)

End Sub

Private Sub print_Click(sender As Object, e As RoutedEventArgs)
    Dim rand As New Random
    Dim randomNumber As Integer = rand.Next()
    Dim randomInRange As Integer = rand.Next(1, 1000)

    Dim filePath As String = $"D:\VB_THESIS_WPS\Prints\Productlist_{randomInRange}.pdf"


    If cashierDataGrid.ItemsSource Is Nothing OrElse cashierDataGrid.Items.Count = 0 OrElse
cashierDataGrid.Columns.Count = 0 Then
```

```
        MessageBox.Show("No data available to print.", "Print Cancelled", MessageBoxButton.OK, MessageBoxImage.Error)
        Return
    End If

    PrintToPDF(cashierDataGrid, filePath)
End Sub
Private Sub PrintToPDF(dataGrid As DataGrid, filePath As String)
    Using pdfWriter As New PdfWriter(filePath)
        Using pdfDocument As New PdfDocument(pdfWriter)
            Dim document As New Document(pdfDocument)

            ' Logo data picture
            Dim logoPath As String = "D:\VB_THESIS_WPS\Images\logo.png"
            Dim logo As Image = New Image(ImageDataFactory.Create(logoPath))
            logo.SetHorizontalAlignment(HorizontalAlignment.Center)
            logo.SetWidth(150)
            document.Add(logo)

            'Address data paragraph
            Dim address As String = "Pamela Mabulay Footwear Retail Store" & vbCrLf &
                                    "#725 Quezon Blvd, Zone 030 Brgy. 308 Quiapo" & vbCrLf &
                                    "Manila, Philippines"
            Dim addressParagraph As New Paragraph(address)
            addressParagraph.SetTextAlignment(TextAlignment.CENTER)
            addressParagraph.SetFontSize(12)
            document.Add(addressParagraph)
            document.Add(New Paragraph().SetHeight(20))
```

```
        'Printed data paragraph


        Dim printedDate As String = DateTime.Today.ToString("yyyy-MM-dd",
CultureInfo.InvariantCulture)
        Dim concatDate As String = $"Date: {printedDate}"
        Dim printedDateParagraph As New Paragraph(concatDate)
        printedDateParagraph.SetTextAlignment(TextAlignment.LEFT)
        printedDateParagraph.SetFontSize(12)
        document.Add(printedDateParagraph)
        document.Add(New Paragraph().SetHeight(10))


        Dim table As New Table(dataGrid.Columns.Count - 2)
        table.SetWidth(UnitValue.CreatePercentValue(100))
        For Each column As DataGridColumn In dataGrid.Columns
            If Not TypeOf column Is DataGridTemplateColumn Then
                table.AddHeaderCell(New Cell().Add(New Paragraph(column.Header.ToString())))
            End If
        Next
        For Each item In dataGrid.ItemsSource
            Dim sale As Cashier = CType(item, Cashier)
            table.AddCell(New Cell().Add(New Paragraph(sale.CashierID.ToString())))
            table.AddCell(New Cell().Add(New Paragraph(sale.Username.ToString())))
            table.AddCell(New Cell().Add(New Paragraph(sale.FirstName.ToString())))
            table.AddCell(New Cell().Add(New Paragraph(sale.LastName.ToString())))
            table.AddCell(New Cell().Add(New Paragraph(sale.Email.ToString())))
            table.AddCell(New Cell().Add(New
Paragraph(String.Format(CultureInfo.InvariantCulture, "{0:yyyy-MM-dd}", sale.CreatedAt))))
        Next
```

```
            document.Add(table)
            document.Close()
            MessageBox.Show("PDF created successfully: ", "Success", MessageBoxButton.OK,
MessageBoxImage.Information)
        End Using
    End Using
    Process.Start(New ProcessStartInfo(filePath) With {
    .UseShellExecute = True
})
End Sub
```

ADD EMPLOYEE

```
Private Sub Insert_Click(sender As Object, e As RoutedEventArgs)
        Dim user As String = Username.Text
        Dim passwords As String = Password.Password
        Dim firstNames As String = Firstname.Text
        Dim lastNames As String = Lastname.Text
        Dim emails As String = Email.Text
        If String.IsNullOrWhiteSpace(user) OrElse String.IsNullOrWhiteSpace(passwords) _
            OrElse String.IsNullOrWhiteSpace(firstNames) OrElse
String.IsNullOrWhiteSpace(lastNames) _
            OrElse String.IsNullOrWhiteSpace(emails) Then
          MessageBox.Show("All fields are required.", "Validation Error", MessageBoxButton.OK,
MessageBoxImage.Warning)
            Return
        End If

        Dim saltBytes As Byte() = GenerateSalt(16)
        Dim hash As Byte() = HashPassword(passwords, saltBytes)

        InsertCashier(user, hash, saltBytes, firstNames, lastNames, emails)
    End Sub

    Private Function GenerateSalt(size As Integer) As Byte()
        Using rng As New RNGCryptoServiceProvider()
            Dim saltBytes(size - 1) As Byte
            rng.GetBytes(saltBytes)
            Return saltBytes
        End Using
```

```
    End Function

    Private Function HashPassword(password As String, salt As Byte()) As Byte()
        Using sha256 As SHA256 = SHA256.Create()
            ' Combine password and salt
            Dim passwordWithSalt As Byte() =
Encoding.UTF8.GetBytes(password).Concat(salt).ToArray()
            ' Compute hash
            Return sha256.ComputeHash(passwordWithSalt)
        End Using
    End Function

    Public Function InsertCashier(username As String, passwordHash As Byte(), salt As Byte(),
firstName As String, lastName As String, email As String)



        Dim query As String = "INSERT INTO [dbo].[Cashier] ([Username], [PasswordHash], [Salt],
[FirstName], [LastName], [Email]) " &
                              "VALUES (@Username, @PasswordHash, @Salt, @FirstName, @LastName,
@Email)"


        Using connection As New SqlConnection(cons.connectionString)
            Using command As New SqlCommand(query, connection)
                ' Use SqlParameter with specific types and sizes
                command.Parameters.Add(New SqlParameter("@Username", SqlDbType.NVarChar,
50)).Value = username
                command.Parameters.Add(New SqlParameter("@PasswordHash",
SqlDbType.VarBinary, passwordHash.Length)).Value = passwordHash
                command.Parameters.Add(New SqlParameter("@Salt", SqlDbType.VarBinary,
salt.Length)).Value = salt
```

```
            command.Parameters.Add(New SqlParameter("@FirstName", SqlDbType.NVarChar,
50)).Value = firstName

            command.Parameters.Add(New SqlParameter("@LastName", SqlDbType.NVarChar,
50)).Value = lastName

            command.Parameters.Add(New SqlParameter("@Email", SqlDbType.NVarChar,
100)).Value = email


            Try

                connection.Open()

                command.ExecuteNonQuery()

                MessageBox.Show("Cashier record inserted successfully!", "Success",
MessageBoxButton.OK, MessageBoxImage.Information)

                emp.FetchCashierData()


                clear()

                Me.Close()

                Return True

            Catch ex As Exception

                MessageBox.Show("Username or email already exist, Please input unique value ",
"Error", MessageBoxButton.OK, MessageBoxImage.Error)

                Return False

            End Try

        End Using

    End Using


End Function
```

EDIT EMPLOYEE

```
  Private Sub Refresh()
      load.FetchCashierData()
    End Sub
    Private Sub Update_Click(sender As Object, e As RoutedEventArgs)
      Dim cashierIDs As Integer = Integer.Parse(editID)
      Dim usernames As String = Username.Text
      Dim firstNames As String = Firstname.Text
      Dim lastNames As String = Lastname.Text
      Dim emails As String = Email.Text



      ' Validation check: ensure none of the fields are empty
      If String.IsNullOrWhiteSpace(usernames) Then
          MessageBox.Show("Username cannot be empty.", "Validation Error",
MessageBoxButton.OK, MessageBoxImage.Warning)
          Username.Focus() ' Set focus back to the Username field
          Return
      End If


      If String.IsNullOrWhiteSpace(firstNames) Then
          MessageBox.Show("First name cannot be empty.", "Validation Error",
MessageBoxButton.OK, MessageBoxImage.Warning)
          Firstname.Focus() ' Set focus back to the Firstname field
          Return
      End If


      If String.IsNullOrWhiteSpace(lastNames) Then
```

```
        MessageBox.Show("Last name cannot be empty.", "Validation Error",
MessageBoxButton.OK, MessageBoxImage.Warning)

        Lastname.Focus() ' Set focus back to the Lastname field

        Return

    End If


    If String.IsNullOrWhiteSpace(emails) Then

        MessageBox.Show("Email cannot be empty.", "Validation Error", MessageBoxButton.OK,
MessageBoxImage.Warning)

        Email.Focus() ' Set focus back to the Email field

        Return

    End If


    UpdateCashier(cashierIDs, usernames, firstNames, lastNames, emails)


End Sub


Public Sub UpdateCashier(cashierID As Integer, username As String, firstName As String,
lastName As String, email As String)

    Dim query As String = "UPDATE [dbo].[Cashier] SET [Username] = @Username,
[FirstName] = @FirstName, [LastName] = @LastName, [Email] = @Email " &

                "WHERE [CashierID] = @CashierID"

    Try

        Using connection As New SqlConnection(con.connectionString)

            Using command As New SqlCommand(query, connection)

                command.Parameters.Add(New SqlParameter("@Username", SqlDbType.NVarChar,
50)).Value = username

                command.Parameters.Add(New SqlParameter("@FirstName", SqlDbType.NVarChar,
50)).Value = firstName

                command.Parameters.Add(New SqlParameter("@LastName", SqlDbType.NVarChar,
50)).Value = lastName
```

```
            command.Parameters.Add(New SqlParameter("@Email", SqlDbType.NVarChar,
100)).Value = email

            command.Parameters.Add(New SqlParameter("@CashierID", SqlDbType.Int)).Value
= editID


            connection.Open()

            Dim rowsAffected As Integer = command.ExecuteNonQuery()

            If rowsAffected > 0 Then

                MessageBox.Show("Cashier record updated successfully!", "Success",
MessageBoxButton.OK, MessageBoxImage.Information)

                Refresh()

                Me.Close()

            Else

                MessageBox.Show("No record found to update.", "Warning",
MessageBoxButton.OK, MessageBoxImage.Warning)


            End If

        End Using

    End Using


    Catch ex As Exception

        MessageBox.Show("The username is already exist, please provide unique username.",
"Error", MessageBoxButton.OK, MessageBoxImage.Error)


    End Try


End Sub
```

ADD PRODUCT

```
Private Sub Insert_Click(sender As Object, e As RoutedEventArgs)

    Dim pname As String = ProductName.Text
    Dim descrip As String = Description.Text
    Dim brands As String = Brand.Text
    Dim Sizes As String = Size.Text
    Dim Colors As String = Color.Text
    Dim prices As Decimal
    If Not Decimal.TryParse(Price.Text, prices) Then
        MessageBox.Show("Please enter a valid decimal value for the price.", "Validation Error", MessageBoxButton.OK, MessageBoxImage.Warning)
        Return
    End If
    If String.IsNullOrWhiteSpace(pname) OrElse String.IsNullOrWhiteSpace(descrip) _
        OrElse String.IsNullOrWhiteSpace(brands) OrElse String.IsNullOrWhiteSpace(Colors) _
        OrElse String.IsNullOrWhiteSpace(Sizes) Then
        MessageBox.Show("All fields are required.", "Validation Error", MessageBoxButton.OK, MessageBoxImage.Warning)
        Return
    End If
    Dim selectedCategory = CType(ComboCat.SelectedItem, Category)
    Dim selectedCatID As Integer = selectedCategory.CategoryID
    InsertCashier(pname, descrip, selectedCatID, brands, Sizes, Colors, prices)


End Sub
Public Sub FetchCategory()
```

```
        Dim query As String = "SELECT CategoryID, CategoryName FROM Category"
        ComboCat.Items.Clear()
        Using cons As New SqlConnection(con.connectionString)
            Using cmd As New SqlCommand(query, cons)
                cons.Open()
                Using reader As SqlDataReader = cmd.ExecuteReader()
                    While reader.Read()
                        Dim category As New Category() With {
                        .CategoryID = reader("CategoryID"),
                        .CategoryName = reader("CategoryName").ToString()
                    }
                        ComboCat.Items.Add(category)
                    End While
                End Using
            End Using
        End Using
    End Sub
    Public Function InsertCashier(pname As String, descrip As String, catIndex As Integer, brands As String, Sizes As String, Colors As String, prices As Decimal)
        Dim productId As Integer
        Dim checkQuery As String = "SELECT COUNT(*) FROM [dbo].[Product] WHERE ProductName = @ProductName"

        Dim insertQuery As String = "INSERT INTO [dbo].[Product] ([ProductName], [Description], [CategoryID], [Brand], [Size], [Color], [Price]) " &
                                    "OUTPUT INSERTED.ProductID " &
                                    "VALUES (@ProductName, @Description, @CategoryID, @Brand, @Size, @Color, @Price)"

        Using connection As New SqlConnection(con.connectionString)
```

```
        connection.Open()

        Using transaction As SqlTransaction = connection.BeginTransaction()
            Try
                ' Check for existing product
                Using checkCommand As New SqlCommand(checkQuery, connection, transaction)
                    checkCommand.Parameters.Add(New SqlParameter("@ProductName",
SqlDbType.NVarChar, 150)).Value = pname
                    Dim count As Integer = Convert.ToInt32(checkCommand.ExecuteScalar())

                    If count > 0 Then
                        MessageBox.Show("A product with the same name already exists.", "Duplicate
Entry", MessageBoxButton.OK, MessageBoxImage.Error)
                        Return pname
                    End If
                End Using

                ' Insert into Product table
                Using insertCommand As New SqlCommand(insertQuery, connection, transaction)
                    insertCommand.Parameters.Add(New SqlParameter("@ProductName",
SqlDbType.NVarChar, 150)).Value = pname
                    insertCommand.Parameters.Add(New SqlParameter("@Description",
SqlDbType.NVarChar, 500)).Value = descrip
                    insertCommand.Parameters.Add(New SqlParameter("@CategoryID",
SqlDbType.Int, 8)).Value = catIndex
                    insertCommand.Parameters.Add(New SqlParameter("@Brand",
SqlDbType.NVarChar, 100)).Value = brands
                    insertCommand.Parameters.Add(New SqlParameter("@Size",
SqlDbType.NVarChar, 50)).Value = Sizes
                    insertCommand.Parameters.Add(New SqlParameter("@Color",
SqlDbType.NVarChar, 50)).Value = Colors
```

```
                insertCommand.Parameters.Add(New SqlParameter("@Price",
SqlDbType.Decimal)).Value = prices
                insertCommand.Parameters("@Price").Precision = 18
                insertCommand.Parameters("@Price").Scale = 2

                productId = Convert.ToInt32(insertCommand.ExecuteScalar())
                MessageBox.Show("Product record inserted successfully!", "Success",
MessageBoxButton.OK, MessageBoxImage.Information)
            End Using

            ' Insert into Inventory table
            InsertToInventory(productId, connection, transaction)

            ' Commit the transaction
            transaction.Commit()

        Catch ex As Exception
            ' Rollback if any error occurs
            transaction.Rollback()
            MessageBox.Show("Transaction failed: " & ex.Message, "Error",
MessageBoxButton.OK, MessageBoxImage.Error)
        End Try
      End Using
    End Using


    inv.FetchProductData()
    Me.Hide()

  End Function
```

```
    Private Sub InsertToInventory(productId As Integer, connection As SqlConnection, transaction
As SqlTransaction)


        Dim qty As Integer = Convert.ToInt32(Quantity.Text)


        Dim q As String = "INSERT INTO Inventory (ProductID, Quantity, OriginalStock,
LastUpdated) " &
                          "VALUES (@ProductID, @Quantity, @OriginalStock, @LastUpdated)"
        Using command As New SqlCommand(q, connection, transaction)
            command.Parameters.Add(New SqlParameter("@ProductID", SqlDbType.Int)).Value =
productId
            command.Parameters.Add(New SqlParameter("@Quantity", SqlDbType.Int)).Value = qty
            command.Parameters.Add(New SqlParameter("@OriginalStock", SqlDbType.Int)).Value =
qty
            command.Parameters.Add(New SqlParameter("@LastUpdated",
SqlDbType.DateTime)).Value = DateTime.Now
            command.ExecuteNonQuery()
        End Using
    End Sub
    Private Sub Clear_Click(sender As Object, e As RoutedEventArgs)


    End Sub
```

UPDATE PRODUCT

```
Public Sub FetchCategory()
    Dim query As String = "SELECT CategoryID, CategoryName FROM Category"
    ComboCat.Items.Clear()
    Using con As New SqlConnection(cons.connectionString)
        Using cmd As New SqlCommand(query, con)
            con.Open()
            Using reader As SqlDataReader = cmd.ExecuteReader()
                While reader.Read()
                    Dim category As New Category() With {
                    .CategoryID = reader("CategoryID"),
                    .CategoryName = reader("CategoryName").ToString()
                }
                    ComboCat.Items.Add(category)
                End While
            End Using
        End Using
    End Using
End Sub
Private Sub ComboCat_MouseDoubleClick(sender As Object, e As MouseButtonEventArgs)
    FetchCategory()
End Sub
Private Sub Update_Click(sender As Object, e As RoutedEventArgs)
    Dim pname As String = ProductName.Text
    Dim descrip As String = Description.Text
    Dim brands As String = Brand.Text
    Dim sizes As String = Size.Text
    Dim colors As String = Color.Text
```

```
        Dim prices As Decimal
        Dim catID As Integer
        If Not Decimal.TryParse(Price.Text, prices) Then
            MessageBox.Show("Please enter a valid decimal value for the price.", "Validation Error",
MessageBoxButton.OK, MessageBoxImage.Warning)
            Return
        End If
        If String.IsNullOrWhiteSpace(pname) OrElse String.IsNullOrWhiteSpace(descrip) _
            OrElse String.IsNullOrWhiteSpace(brands) OrElse String.IsNullOrWhiteSpace(colors) _
            OrElse String.IsNullOrWhiteSpace(sizes) Then
            MessageBox.Show("All fields are required.", "Validation Error", MessageBoxButton.OK,
MessageBoxImage.Warning)
            Return
        End If
        If ComboCat.SelectedItem Is Nothing Then
            MessageBox.Show("Please select a category.", "Warning", MessageBoxButton.OK,
MessageBoxImage.Warning)
            Return
        Else
            If TypeOf ComboCat.SelectedItem Is Category Then
                catID = CType(ComboCat.SelectedItem, Category).CategoryID
            ElseIf TypeOf ComboCat.SelectedItem Is String Then
                Dim selectedCategory As String = ComboCat.SelectedItem.ToString()
                Dim category As Category = FetchCategoryByName(selectedCategory)
                If category IsNot Nothing Then
                    catID = category.CategoryID
                Else
                    MessageBox.Show("Selected category does not exist.", "Warning",
MessageBoxButton.OK, MessageBoxImage.Warning)
                    Return
```

```
            End If
        End If
    End If
    UpdateProduct(pname, descrip, brands, sizes, colors, prices, catID)
End Sub
Public Sub UpdateProduct(pname As String, descrip As String, brands As String, sizes As String, colors As String, price As Double, catID As Integer)

    Dim query As String = "UPDATE [dbo].[Product] SET [ProductName] = @ProductName, [Description] = @Description, [Brand] = @Brand, [Size] = @Size, [Color] = @Color,  [Price] = @Price, [CategoryID] = @CategoryID " &
                          "WHERE [ProductID] = @ProductID"

    Using connection As New SqlConnection(cons.connectionString)
        Using command As New SqlCommand(query, connection)
            command.Parameters.Add(New SqlParameter("@ProductName", SqlDbType.NVarChar, 50)).Value = pname
            command.Parameters.Add(New SqlParameter("@Description", SqlDbType.NVarChar, 50)).Value = descrip
            command.Parameters.Add(New SqlParameter("@Brand", SqlDbType.NVarChar, 100)).Value = brands
            command.Parameters.Add(New SqlParameter("@Size", SqlDbType.NVarChar, 50)).Value = sizes
            command.Parameters.Add(New SqlParameter("@Color", SqlDbType.NVarChar, 50)).Value = colors
            command.Parameters.Add(New SqlParameter("@Price", SqlDbType.Decimal)).Value = price
            command.Parameters("@Price").Precision = 18
            command.Parameters("@Price").Scale = 2
            command.Parameters.Add(New SqlParameter("@CategoryID", SqlDbType.Int)).Value = catID
```

```
            command.Parameters.Add(New SqlParameter("@ProductID", SqlDbType.Int)).Value =
editID

            connection.Open()

            Dim rowsAffected As Integer = command.ExecuteNonQuery()

            If rowsAffected > 0 Then

                MessageBox.Show("Product record updated successfully!", "Success",
MessageBoxButton.OK, MessageBoxImage.Information)

                Clear()

                RefreshForm()

                Me.Close()

            Else

                MessageBox.Show("No record found to update.", "Warning", MessageBoxButton.OK,
MessageBoxImage.Warning)

            End If

        End Using

    End Using

End Sub

Public Function FetchCategoryByName(catName As String) As Category

    Dim query As String = "SELECT CategoryID, CategoryName FROM Category WHERE
CategoryName = @CategoryName"

    Dim fetchedCategory As Category = Nothing

    Using con As New SqlConnection(cons.connectionString)

        Using cmd As New SqlCommand(query, con)

            cmd.Parameters.Add(New SqlParameter("@CategoryName", SqlDbType.NVarChar,
50)).Value = catName

            con.Open()


            Using reader As SqlDataReader = cmd.ExecuteReader()

                If reader.Read() Then

                    fetchedCategory = New Category() With {
```

```
                .CategoryID = reader("CategoryID"),
                .CategoryName = reader("CategoryName").ToString()
            }
            End If
        End Using
    End Using
End Using

Return fetchedCategory
End Function
```

VIEW PRODUCT

```
Dim colorHexArray(,) As String = {
   {"Red", "#FF5252"},
   {"Blue", "#1E88E5"},
   {"Green", "#0CA678"},
   {"Yellow", "#FFC107"},
   {"Orange", "#FF8F00"},
   {"Purple", "#8E24AA"},
   {"Teal", "#00796B"},
   {"Brown", "#6D4C41"},
   {"Gray", "#9E9E9E"},
   {"Black", "#000000"},
   {"White", "#FFFFFF"},
   {"Pink", "#C2185B"}
}

   Public Sub New(Id As Integer, pname As String, prices As Double, descrip As String, catname
As String, brands As String, sizes As String, colors As String)


      InitializeComponent()

      productname.Text = pname
      Price.Text = prices
      description.Text = descrip
      category.Text = catname
      brand.Text = brands
      size.Text = sizes
      editID = Id
      currentColor = colors



      Dim hexCode As String = GetHexCodeByColor(currentColor)
      If hexCode IsNot Nothing Then

         Dim colorBrush As SolidColorBrush = CType((New
BrushConverter().ConvertFromString(hexCode)), SolidColorBrush)
         ColorBorder.Background = colorBrush
         BorderColorOutline.BorderBrush = colorBrush
         '  productname.Foreground = colorBrush
         ' size.Foreground = colorBrush
         ' description.Foreground = colorBrush
         ' Price.Foreground = colorBrush
         ' brand.Foreground = colorBrush
```

```
            ' category.Foreground = colorBrush
            ' Price_Copy.Foreground = colorBrush
        Else
            MessageBox.Show("Color not found in the array.", "Error", MessageBoxButton.OK,
MessageBoxImage.Error)
        End If
        FetchImagePath()

    End Sub
    Private Sub btnClose_Click(sender As Object, e As RoutedEventArgs)
        Me.Close()
    End Sub


    Private Sub FetchImagePath()

        Dim query As String = "SELECT ImageUrl FROM ProductImage Where ProductID=
@ProductID"
        Using connection As New SqlConnection(connectionString)

            connection.Open()

            Using cmd As New SqlCommand(query, connection)


                cmd.Parameters.AddWithValue("@ProductID", editID)

                Using reader As SqlDataReader = cmd.ExecuteReader()

                    If reader.Read() Then
                        Dim imageUrl As String = reader("ImageUrl").ToString()

                        SetImageSource(imageUrl)

                    End If
                End Using



            End Using


        End Using
    End Sub


    Private Sub SetImageSource(imageUrl As String)
```

```
    Dim defaultImg As String = "D:\VB_THESIS_WPS\Images\logo.png"
    Dim bitmap As New BitmapImage()

    If Not String.IsNullOrEmpty(imageUrl) Then
        Try
            bitmap.BeginInit()
            bitmap.UriSource = New Uri(imageUrl, UriKind.RelativeOrAbsolute)
            bitmap.EndInit()
            Imagepreview.Source = bitmap
        Catch ex As Exception
            bitmap = New BitmapImage(New Uri(defaultImg, UriKind.RelativeOrAbsolute))
            Imagepreview.Source = bitmap
        End Try
    Else
        bitmap = New BitmapImage(New Uri(defaultImg, UriKind.RelativeOrAbsolute))
        Imagepreview.Source = bitmap
    End If
End Sub


Private Function GetHexCodeByColor(colorName As String) As String
    For i As Integer = 0 To colorHexArray.GetLength(0) - 1
        If colorHexArray(i, 0).Equals(colorName, StringComparison.OrdinalIgnoreCase) Then
            Return colorHexArray(i, 1)
        End If
    Next
    Return Nothing
End Function
```

STOCKIN

```
  Private Sub RefreshForm()

    load.FetchProductData()
  End Sub
  Private Sub EditStock()
    If String.IsNullOrWhiteSpace(newstock.Text) Then
      MessageBox.Show("Stock value cannot be empty.", "Error", MessageBoxButton.OK,
MessageBoxImage.Error)
      Return
    End If
    Dim newStocks As Integer = Integer.Parse(newstock.Text)
    Dim currentQuantity As Integer


    If newStocks = 0 Then

      MessageBox.Show("You can't update stock using 0 value", "Error", MessageBoxButton.OK,
MessageBoxImage.Error)

      Return

    End If
    Dim selectQuery As String = "SELECT Quantity FROM Inventory WHERE InventoryID =
@InventoryID"

    Using mycon As New SqlConnection(connectionString)
      mycon.Open()

      Using selectCmd As New SqlCommand(selectQuery, mycon)
        selectCmd.Parameters.AddWithValue("@InventoryID", editID)

        Dim reader As SqlDataReader = selectCmd.ExecuteReader()

        If reader.Read() Then
          currentQuantity = Convert.ToInt32(reader("Quantity"))
        Else
          MessageBox.Show("No record found for the given InventoryID.", "Error",
MessageBoxButton.OK, MessageBoxImage.Error)
          Exit Sub
        End If
        reader.Close()
      End Using
```

```
        Dim updateQuery As String = "UPDATE Inventory SET Quantity = Quantity + @Quantity,
OriginalStock = @OriginalStock WHERE InventoryID = @InventoryID"

        Using updateCmd As New SqlCommand(updateQuery, mycon)
            updateCmd.Parameters.AddWithValue("@Quantity", newStocks)
            updateCmd.Parameters.AddWithValue("@OriginalStock", currentQuantity + newStocks)
            updateCmd.Parameters.AddWithValue("@InventoryID", editID)

            updateCmd.ExecuteNonQuery()
        End Using
    End Using

    MessageBox.Show("Updated successfully", "Success", MessageBoxButton.OK,
MessageBoxImage.Information)
    RefreshForm()
    Me.Close()
                                        End Sub
```

PRODUCTIMAGE

```
  Private Sub RefreshForm()

    load.FetchProductData()
  End Sub
  Private Sub EditStock()
    If String.IsNullOrWhiteSpace(newstock.Text) Then
      MessageBox.Show("Stock value cannot be empty.", "Error", MessageBoxButton.OK,
MessageBoxImage.Error)
      Return
    End If
    Dim newStocks As Integer = Integer.Parse(newstock.Text)
    Dim currentQuantity As Integer


    If newStocks = 0 Then

      MessageBox.Show("You can't update stock using 0 value", "Error", MessageBoxButton.OK,
MessageBoxImage.Error)

      Return

    End If
    Dim selectQuery As String = "SELECT Quantity FROM Inventory WHERE InventoryID =
@InventoryID"

    Using mycon As New SqlConnection(connectionString)
      mycon.Open()

      Using selectCmd As New SqlCommand(selectQuery, mycon)
        selectCmd.Parameters.AddWithValue("@InventoryID", editID)

        Dim reader As SqlDataReader = selectCmd.ExecuteReader()

        If reader.Read() Then
          currentQuantity = Convert.ToInt32(reader("Quantity"))
        Else
          MessageBox.Show("No record found for the given InventoryID.", "Error",
MessageBoxButton.OK, MessageBoxImage.Error)
          Exit Sub
        End If
        reader.Close()
      End Using
```

```
        Dim updateQuery As String = "UPDATE Inventory SET Quantity = Quantity + @Quantity,
OriginalStock = @OriginalStock WHERE InventoryID = @InventoryID"

        Using updateCmd As New SqlCommand(updateQuery, mycon)
            updateCmd.Parameters.AddWithValue("@Quantity", newStocks)
            updateCmd.Parameters.AddWithValue("@OriginalStock", currentQuantity + newStocks)
            updateCmd.Parameters.AddWithValue("@InventoryID", editID)

            updateCmd.ExecuteNonQuery()
        End Using
    End Using

    MessageBox.Show("Updated successfully", "Success", MessageBoxButton.OK,
MessageBoxImage.Information)
    RefreshForm()
    Me.Close()
                                        End Sub
```

INSERT IMAGE

```
Private Sub btnBrowseImage_Click(sender As Object, e As RoutedEventArgs)
    Dim openFileDialog As New OpenFileDialog()

    openFileDialog.Filter = "Image files (*.jpg;*.jpeg;*.png;*.bmp)|*.jpg;*.jpeg;*.png;*.bmp|All
files (*.*)|*.*"

    openFileDialog.Title = "Select an Image File"

    If openFileDialog.ShowDialog() = True Then
        Dim selectedFilePath As String = openFileDialog.FileName

        txtImagePath.Text = selectedFilePath

        Dim bitmap As New BitmapImage(New Uri(selectedFilePath))
        imgPreview.Source = bitmap
    End If
End Sub

Private Sub updatebtn_Click(sender As Object, e As RoutedEventArgs) Handles updatebtn.Click
    Dim selectedCategory = CType(ComboCat.SelectedItem, Product)

    If selectedCategory Is Nothing OrElse selectedCategory.ProductID = 0 Then
        MessageBox.Show("Please select a valid category.", "Error", MessageBoxButton.OK,
MessageBoxImage.Error)
        Return
    End If



    Dim selectedCatID As Integer = selectedCategory.ProductID
    Dim path As String = txtImagePath.Text




    InsertOrUpdateProductImage(selectedCatID, path)
End Sub

Public Sub InsertOrUpdateProductImage(id As Integer, uploadedImage As String)
    If String.IsNullOrEmpty(txtImagePath.Text) Then
        MessageBox.Show("Please choose a file before inserting.", "Insert Failed",
MessageBoxButton.OK, MessageBoxImage.Error)
    Else
        ' Use the Documents folder to create a safe path for ProductImages
```

```
        Dim imageFolder As String =
IO.Path.Combine(Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments),
"VB_THESIS_WPS\ProductImages")

        ' Check if the directory exists, create it if it doesn't
        If Not IO.Directory.Exists(imageFolder) Then
            IO.Directory.CreateDirectory(imageFolder)
        End If


        Dim fileName As String = Path.GetFileName(uploadedImage)
        Dim uniqueFileName As String = Guid.NewGuid().ToString() & "_" & fileName
        Dim imagePath As String = Path.Combine(imageFolder, uniqueFileName)

        File.Copy(uploadedImage, imagePath, overwrite:=True)

        Dim query As String = "
        IF EXISTS (SELECT 1 FROM ProductImage WHERE ProductID = @ProductID)
            UPDATE ProductImage
            SET ImageUrl = @ImageUrl
            WHERE ProductID = @ProductID
        ELSE
            INSERT INTO ProductImage (ProductID, ImageUrl)
            VALUES (@ProductID, @ImageUrl)
    "

        Using connection As New SqlConnection(connectionString)
            Using command As New SqlCommand(query, connection)
                command.Parameters.AddWithValue("@ProductID", id)
                command.Parameters.AddWithValue("@ImageUrl", imagePath)

                connection.Open()
                command.ExecuteNonQuery()
            End Using
        End Using

        MessageBox.Show("Image inserted or updated successfully!", "Success",
MessageBoxButton.OK, MessageBoxImage.Information)
        Refresh()
        ComboCat.SelectedItem = Nothing
        txtImagePath.Text = ""
        imgPreview.Source = Nothing
    End If
End Sub

Public Sub FetchCategory()
    Dim query As String = "SELECT ProductID, ProductName FROM Product"
```

```
    ComboCat.Items.Clear()

    Using cons As New SqlConnection(connectionString)
        Using cmd As New SqlCommand(query, cons)
            cons.Open()

            Using reader As SqlDataReader = cmd.ExecuteReader()
                While reader.Read()
                    Dim category As New Product() With {
                    .ProductID = reader("ProductID"),
                    .ProductName = reader("ProductName").ToString()
                }
                    ComboCat.Items.Add(category)
                End While
            End Using
        End Using
    End Using
End Sub

Private Sub btnClose_Click(sender As Object, e As RoutedEventArgs) Handles btnClose.Click
    Me.Close()
End Sub
Private Sub Refresh()
    imageform.FetchImages()

                                                    End Sub
```

Installation Guide

Pre-Installation Steps

1. Install .Net Framework
    - Download and install .Net Framework 4.8
2. Install Microsoft SQL Server
    - Download and Install Microsoft SQL Server 2022 or Later
3. Install SQL Server Management Studio (SSMS)
    - Download and install SQL Server Management Studio (SSMS) to manage the database.
4. Create Database
    - Open SSMS and log in using the credentials you set during SQL Server installation.
    - Execute the provided `.sql` file (e.g., `Pamela.sql`) or `.bak` file (e.g., `Pamela.bak`) to set up the database schema and seed data.

Installation Steps

1. Download the System Installer
    - Obtain the setup files from the project source or installation package.

2. Extract the Files
    - After downloading the project select the `.zip` file and extract the files

3. Configure Database Connection
    - Open the `App.config` file in the system's installation directory.
    - Update the `<connectionStrings>` section with your SQL Server details

4. Install Prerequisite Packages
    - If required, install dependencies like Microsoft Visual C++ Redistributables.

Post-Installation Steps

1. Test Database Connection
    - Launch the system and log in using the default admin credentials provided in the documentation.
2. Set Up Initial Configuration
    - Configure initial settings (e.g., product details, inventory, user roles) using the admin dashboard.
3. Verify System Functions
    - Check key features like inventory updates, sales transactions, and reporting to ensure proper functionality.

Uninstallation

1. Remove Application
    - Navigate to the System Folder, hit right-click with the mouse and click delete
2. Delete Database
    - Open SSMS, locate the `pamela` database, and delete it if no longer needed.
3. Clean Up Files
    - Delete any leftover files from the installation directory.